LocalHub — Master Development Prompt

Engineering Specification & AI Operating Manual

Document Classification: INTERNAL ENGINEERING SPECIFICATION
Project Codename: LocalHub
Repository: `https://github.com/lastergamer8228-spec/LocalHub`
Package Namespace: `com.localhub.localhub`
Target Platform: Android (Min SDK 24 • Target/Compile SDK 34 • JVM 17)
Language: Kotlin (1.9.24) with Jetpack Compose (Material 3) + Vanilla JavaScript/CSS for Browser UI
Build System: Gradle 8.2 with KSP
Document Version: 1.0.0

PREAMBLE: READER INSTRUCTIONS

This document is the singular source of truth for all future development on the LocalHub project. It is not a suggestion document. It is not a wishlist. It is a binding engineering specification.

Before you modify, add, remove, or refactor any line of code in this project, you must:

1. **Read the entire repository.** Every file. Every class. Every function. You cannot make correct decisions about a system you do not fully understand.
2. **Read this document in its entirety.** The rules in Section 3 are non-negotiable. The architecture in Section 5 is sacred. The patterns described herein are intentional and proven.
3. **Understand the dependency graph.** Every component exists within a web of relationships. Changing one node affects others. Map your changes before writing them.
4. **Verify no duplicate exists.** Before creating anything new, confirm that an equivalent implementation does not already exist somewhere in the codebase. Reuse is mandatory.
5. **Prefer extension over replacement.** Working systems stay. New capability attaches to existing interfaces. The bar for rewriting a working module is impossibly high — it must be broken beyond repair, not merely imperfect.

If you are uncertain whether a change is safe, it is not safe. Ask. Document. Iterate. Do not guess.

TABLE OF CONTENTS

1. Project Philosophy
2. Project Vision
3. Architecture Rules & Invariants
4. Rules That Must Never Be Broken

5. System Architecture Deep Dive
6. Android Application Responsibilities
7. Browser UI Responsibilities
8. Server Responsibilities
9. Assistant Architecture
10. Brain Architecture
11. Context Engine
12. Intent Engine
13. Capability Registry
14. Task Planner
15. Automation Engine
16. Plugin SDK
17. Memory System
18. File Manager
19. Media Player
20. Dashboard
21. Authentication & Authorization
22. Security Standards
23. REST API Standards
24. WebSocket Standards
25. Module Standards
26. Folder Structure Standards
27. Coding Standards
28. Naming Conventions
29. Documentation Standards
30. Testing Standards
31. Performance Standards
32. UI Standards
33. Animation Standards
34. Accessibility Standards
35. Release Standards
36. Future Expansion Rules
37. How New Modules Should Be Created
38. How Old Modules Should Be Modified
39. How Duplicate Code Should Be Avoided
40. How Future AI Should Think Before Changing Anything

---

1. PROJECT PHILOSOPHY

1.1 The Fundamental Axiom

LocalHub is a device-first, offline-first, privacy-first local server that runs entirely on a single Android phone. Every design decision flows from this axiom. The phone is not a thin client reaching for a cloud. The phone IS the cloud — self-contained, self-sufficient, and sovereign.

1.2 Design Principles (Ordered by Priority)

P0 — Non-Negotiable

Principle	Meaning
Offline-First	The application must function with zero internet connectivity. All features must work over LAN, Wi-Fi Direct, or mobile hotspot. Cloud dependencies are forbidden unless opt-in and clearly labeled.
Deterministic	Given the same inputs and state, the system must produce the same outputs. No randomness in critical paths. No time-dependent behavior without explicit timeouts. No implicit ordering assumptions.
Secure by Default	Every endpoint, every API, every file access must assume hostile input. Authentication gates every action. Sessions expire. Input is validated. Secrets are never hardcoded.
Modular	Every feature exists within a module. Modules are self-contained, independently testable, and independently enable/disable. The system is a collection of cooperating modules, not a monolith.

P1 — Strongly Enforced

Principle	Meaning
Maintainable	Code must be readable by a competent Kotlin developer who has never seen this project before. Complex algorithms require comments. Public APIs require KDoc. Naming must be self-documenting.
Performant	The server must handle concurrent requests on low-end devices without ANRs. The UI must remain at 60fps. Memory usage must be bounded. Background work must not starve foreground tasks.
Scalable	The architecture must support adding new modules, new APIs, new UI screens, and new automation rules without modifying core infrastructure. The cost of extension must be proportional to the complexity of the new feature, not the size of the existing codebase.
Privacy-Respecting	No data leaves the device without explicit user consent. Logs are stored locally. Client information is ephemeral by default. No analytics, no tracking, no telemetry to external servers.

P2 — Important

Principle	Meaning
Testable	Every public class must be testable in isolation. Dependencies are injected. Side effects are controlled. No static global state that cannot be reset in tests.
Accessible	Both the Android UI and the Browser UI must be usable by people with disabilities. Content descriptions, color contrast, keyboard navigation, screen reader support.
Aesthetic	The interface must be beautiful. Dark-first. Clean. Modern. Material 3 on Android. Dark glass aesthetic on the browser. No visual clutter. No decorative chrome.

1.3 What LocalHub Is NOT

- LocalHub is NOT a cloud platform. It will never require an internet connection to function.
- LocalHub is NOT a multi-server system. One phone, one server, one instance.
- LocalHub is NOT a general-purpose web framework. The server exists to serve the device and its LAN clients, not to host arbitrary websites.
- LocalHub is NOT an open relay. Every access is authenticated and authorized.
- LocalHub is NOT a battery hog. The server must be efficient. Background work must be minimized. WakeLocks must be scoped and timed.
- LocalHub is NOT a beginner project. The codebase assumes competence in Kotlin, Android development, HTTP protocols, and clean architecture.

1.4 The Offline-First Contract

Every feature in LocalHub must satisfy the following contract:

```
IF the device has NO internet connectivity AND the device has a working local network i
  nterface:
      THEN the feature MUST still function at full capability.

  IF the device has NO network interface at all:
      THEN the feature MUST still provide local-only capabilities (file management, med
  ia, assistant, etc.).
```

Any feature that violates this contract is architecturally invalid and must be redesigned.

1.5 The Determinism Contract

Every component in LocalHub must behave deterministically:

- The Context Engine must produce the same context state given the same inputs and prior state.
- The Task Planner must produce the same plan given the same task description and available capabilities.
- The Automation Engine must fire the same actions given the same trigger conditions.
- The Memory System must retrieve the same information given the same query and stored data.

- The Intent Engine must classify the same intent given the same input utterance.

Non-determinism is only permitted in: (a) UI animations, (b) random ID generation (UUIDs), (c) timing-dependent operations where the order is inherently unpredictable (network events). All other behavior must be deterministic and reproducible.

---

2. PROJECT VISION

2.1 The Vision Statement

LocalHub envisions a world where every person carries a complete, private, offline-capable local server in their pocket. A server that can serve files, play media, manage devices, automate tasks, answer questions, and provide intelligent assistance — all without depending on any external infrastructure. A server that is secure, fast, beautiful, and extensible.

2.2 The End State (v1.0)

At v1.0, LocalHub will provide:

1. **A rock-solid local HTTP/WebSocket server** that serves system telemetry, logs, and module-contributed content to any browser on the network.
2. **A comprehensive browser dashboard** that visualizes device state, connected clients, and module data in real-time with a beautiful dark glass aesthetic.
3. **A full-featured Android application** with native Compose UI for server management, system monitoring, log inspection, module control, and settings.
4. **An intelligent local assistant** that understands natural language commands, maintains conversational context, executes device actions, manages files, controls media, and automates repetitive tasks — all running entirely on-device with no cloud dependency.
5. **A plugin ecosystem** that allows third-party developers to extend LocalHub with new capabilities through a well-defined SDK.
6. **An automation engine** that responds to triggers (time-based, event-based, state-based) with configurable actions.
7. **A memory system** that persists user preferences, conversation history, and learned patterns across sessions.
8. **Authentication and authorization** with role-based access control (Admin Mode vs Web User Mode).
9. **Real file management** through both the Android app and the browser UI.
10. **Real media control** with playback, queue management, and streaming over the local network.

2.3 The Roadmap

```
Phase 1: STABILIZE (Current)
    ├── Fix all existing bugs
    ├── Optimize server performance
    ├── Complete all REST endpoints
    ├── Polish the browser dashboard
    ├── Harden the module system
    └── Ensure 100% offline operation

  Phase 2: FINISH ASSISTANT
    ├── Brain architecture (reasoning engine)
    ├── Context Engine (conversation + state tracking)
    ├── Intent Engine (NLU classification)
    ├── Capability Registry (action catalog)
    ├── Task Planner (multi-step execution)
    ├── Automation Engine (triggers + actions)
    ├── Plugin Learning (auto-discovery)
    └── Memory System (persistent learning)

  Phase 3: AUTHENTICATION & SECURITY
    ├── Login system (Admin + Web User)
    ├── Session management
    ├── Role-based authorization
    ├── API endpoint protection
    ├── Input sanitization
    └── Rate limiting

  Phase 4: FILE MANAGER & MEDIA
    ├── File browsing (both app + browser)
    ├── File upload/download
    ├── Media player (audio + video)
    ├── Streaming over HTTP
    └── Playlist management

  Phase 5: MULTI-DEVICE TESTING
    ├── Test with 5+ concurrent clients
    ├── Test across Android versions (7.0 - 15.0)
    ├── Test on low-RAM devices (<2GB)
    ├── Test on weak networks (edge-case Wi-Fi)
    └── Stress test WebSocket connections

  Phase 6: POLISH & RC
```

```
    ├── Animation refinement
    ├── Accessibility audit
    ├── Performance profiling
    ├── Documentation completion
    └── Release candidate build

Phase 7: v1.0 RELEASE
```

2.4 Version Strategy

- **v0.x.y** — Pre-release. Architecture may change. APIs may break.
- **v1.0.0** — First stable release. All core features complete. Public API locked.
- **v1.x.y** — Backward-compatible additions. No breaking changes to public APIs.
- **v2.0.0** — Major architecture evolution. Requires migration guide.

---

3. ARCHITECTURE RULES & INVARIANTS

3.1 The Layered Architecture

LocalHub follows a strict layered architecture with unidirectional dependencies. Dependencies flow inward only. Outer layers depend on inner layers. Inner layers never depend on outer layers.



3.2 The Dependency Rule

No layer may import from a layer above it. Specifically:

- `domain/model` must not import `android.*`, `data.*`, `server.*`, `ui.*`, or `infrastructure.*`.
- `data/repository` may import `data/local` (DAOs, entities) and `domain/model` but NOT `server.*` or `ui.*`.
- `server/*` may import `domain/model`, `data/repository`, and `system/*` but NOT `ui/*`.
- `ui/*` may import `domain/model`, `core/*`, and `data/datastore` but NOT `server/*` directly (data flows through the AppContainer's StateFlows).
- `core/AppContainer` is the composition root and may reference everything, but it only constructs and wires — it contains no business logic.

3.3 The Single Responsibility Rule

Every file has exactly one reason to change. Every class has exactly one responsibility. Every function does exactly one thing.

- `LocalHubServer.kt` manages the HTTP server lifecycle and request dispatch.
- `ApiRouter.kt` maps URL paths to handler logic.
- `AssetServer.kt` serves static files from app assets.

- `ClientTracker.kt` tracks connected clients.
- `WebSocketHub.kt` manages WebSocket fan-out.
- `LocalHubWebSocket.kt` handles a single WebSocket connection.
- `SystemInfoProvider.kt` collects device telemetry.
- `AppLogger.kt` logs to Room and SharedFlow.
- `ModuleManager.kt` manages module registration and request dispatch.
- `PreferencesManager.kt` manages DataStore preferences.

This single-responsibility pattern must be preserved. When adding new functionality, create a new class. Do not append logic to an existing class.

3.4 The State Flow Pattern

All reactive state in LocalHub flows through Kotlin `StateFlow` or `SharedFlow`. This is not optional. There are no callbacks, no listeners, no observer interfaces (except where NanoHTTPD's API requires them).

- `StateFlow<T>` for state that has a current value (server status, system info, connected clients).
- `SharedFlow<T>` for events that don't have a current value (log entries, notifications).
- ViewModels expose `StateFlow<T>` to Compose via `collectAsStateWithLifecycle()`.
- The server broadcasts state changes through `WebSocketHub` to all connected browser clients.

3.5 The Repository Pattern

All data access goes through repositories. No ViewModel, no Server component, and no UI component directly accesses a DAO.

- `LogRepository` wraps `LogDao`
- `SettingRepository` wraps `SettingDao`
- `ModuleRepository` wraps `ModuleDao`
- `StatisticsRepository` wraps `StatisticsDao`

New data sources require a new repository. Repositories return domain models, not entities. The mapping between entities and domain models happens in `data/local/Mappers.kt`.

3.6 The Manual DI Pattern

LocalHub uses a **manual dependency container** (`AppContainer`). There is no DI framework (no Hilt, no Dagger, no Koin). This is intentional and must be preserved.

- `AppContainer` is constructed once in `LocalHubApplication.onCreate()`.
- All dependencies are `val` (immutable) properties on `AppContainer`.
- ViewModels access the container through `(getApplication() as LocalHubApplication).appContainer`.
- The server components receive their dependencies through constructor parameters.

If a new component is needed:

1. Create the class with its dependencies as constructor parameters.
2. Instantiate it in `AppContainer` with the appropriate dependencies.
3. Expose it as a `val` property if other components need it.

3.7 The Immutability Preference

Domain models are `data class` with `val` properties. State is updated by creating new instances. The only exceptions are:

- `var isEnabled: Boolean` in `LocalModule` (modules can be toggled at runtime).
- `MutableStateFlow` backing fields (which are encapsulated behind `StateFlow` public API).
- `ClientTracker`'s internal `ConcurrentHashMap` (concurrent access requires mutability, but the public API exposes only `StateFlow`).

3.8 The Concurrency Model

- All I/O operations (Room, DataStore, file operations) run on `Dispatchers.IO`.
- All UI updates happen on `Dispatchers.Main`.
- The server runs on NanoHTTPD's internal thread pool.
- The foreground service uses `CoroutineScope(SupervisorJob() + Dispatchers.IO)`.
- `AppLogger` uses its own `CoroutineScope(SupervisorJob() + Dispatchers.IO)` for fire-and-forget logging.
- WebSocket broadcasts are thread-safe via `CopyOnWriteArrayList`.
- Client tracking is thread-safe via `ConcurrentHashMap`.

---

4. RULES THAT MUST NEVER BE BROKEN

These are the **invariants** of the LocalHub system. They are absolute. No feature request, no deadline, no convenience justifies violating them.

4.1 The Ten Commandments

I. Thou shalt not require internet connectivity for any core feature.

Every feature must work offline. If a feature requires the internet, it is not a LocalHub feature. Period.

II. Thou shalt not break backward compatibility of the REST API without a version bump.

If you add a field to an API response, that's fine. If you remove a field or change its type, that requires a major version bump and a migration guide.

III. Thou shalt not store secrets in source code or assets.

API keys, passwords, tokens, and certificates must never appear in committed code. Secrets are stored in `EncryptedSharedPreferences` or derived at runtime.

IV. Thou shalt not block the main thread.

No synchronous I/O on `Dispatchers.Main`. No `runBlocking` in ViewModel code. No database queries on the main thread. The UI must never stutter.

V. Thou shalt not introduce a DI framework.

The manual `AppContainer` pattern works. It is simple, transparent, and debuggable. Adding Hilt or Dagger would increase complexity for zero benefit at this scale.

VI. Thou shalt not rewrite a working module.

If a module functions correctly and its tests pass, you may extend it. You may not rewrite it. The bar for "it needs to be rewritten" is: "it is fundamentally broken and cannot be fixed by incremental changes."

VII. Thou shalt not duplicate functionality.

Before writing a new function, search the codebase. If an equivalent function exists, use it. If it's close but not quite right, extend it. Duplication is the root of maintenance nightmares.

VIII. Thou shalt not bypass the repository layer.

All data access goes through repositories. ViewModels, server components, and UI components do not import DAOs. Period.

IX. Thou shalt not leak memory.

Every `Flow` subscription must be lifecycle-aware. Every `BroadcastReceiver` must be unregistered. Every `WakeLock` must be released. Every `CoroutineScope` must be cancelled on destruction.

X. Thou shalt not compromise security for convenience.

Input is always validated. Sessions always expire. Endpoints are always authenticated. CORS is always restricted in production. The cost of security is never too high.

4.2 Additional Immutable Constraints

The package structure is fixed. New packages may be added at the same level as existing ones (`assistant`, `automation`, `auth`, etc.), but the existing package hierarchy must not be reorganized.

The module interface is fixed. `LocalModule` defines `id`, `name`, `description`, `version`, `isEnabled`, `start()`, `stop()`, `handleRequest()`, and `toInfo()`. These members must not be removed or renamed. New members may be added with default implementations.

The database schema must be versioned. Every schema change increments the version number and provides a migration. `fallbackToDestructiveMigration()` is acceptable for pre-1.0 but must be replaced with proper migrations before v1.0.

The WebSocket protocol is append-only. New message types may be added. Existing message types must not be changed. The `{"type": "...", "data": {...}}` envelope format is permanent.

Constants must remain in `Constants.kt`. Magic numbers must not be scattered across files. If a value is configurable, it goes in `Constants` or `PreferencesManager`.

---

5. SYSTEM ARCHITECTURE DEEP DIVE

5.1 Component Graph

```
LocalHubApplication (Application)
      │
      └── AppContainer (Manual DI — Composition Root)
          │
          ├── Gson (JSON serialization)
          │
          ├── PreferencesManager (DataStore)
          │   ├── portFlow: Flow<Int>
          │   ├── themeFlow: Flow<String>
          │   ├── autostartFlow: Flow<Boolean>
          │   └── serverNameFlow: Flow<String>
          │
          ├── AppDatabase (Room — localhub.db)
          │   ├── LogDao → LogRepository
          │   ├── SettingDao → SettingRepository
          │   ├── ModuleDao → ModuleRepository
          │   └── StatisticsDao → StatisticsRepository
          │
          ├── AppLogger (Room + SharedFlow)
          │   └── logs: SharedFlow<LogEntry>  (replay=64, extraBuffer=256)
          │
          ├── SystemInfoProvider (Telemetry)
          │   └── lastSnapshot: StateFlow<SystemInfo?>
          │
          ├── ModuleManager (Plugin Registry)
          │   └── modules: CopyOnWriteArrayList<LocalModule>
          │       └── DashboardModule (default)
          │
          ├── ClientTracker (Live Client Registry)
          │   └── clients: StateFlow<List<ConnectedClient>>
          │
          ├── serverStatus: StateFlow<ServerStatus>
          ├── connectedClients: StateFlow<List<ConnectedClient>>
          ├── isServerRunning: MutableStateFlow<Boolean>
          │
          └── LocalHubServer (NanoWSD, port from PreferencesManager)
              ├── ApiRouter (/api/*)
              │   ├── GET /api/status
```

```
│   ├── GET /api/system
│   ├── GET /api/device
│   ├── GET /api/network
│   ├── GET /api/storage
│   ├── GET /api/battery
│   ├── GET /api/cpu
│   ├── GET /api/memory
│   ├── GET /api/modules
│   ├── GET /api/clients
│   ├── GET /api/logs
│   ├── GET /api/statistics
│   ├── POST /api/log
│   └── POST /api/command
│
├── AssetServer (dashboard assets from app/src/main/assets/)
│   ├── / → dashboard/index.html
│   ├── /style.css → dashboard/style.css
│   └── /app.js → dashboard/app.js
│
└── WebSocketHub → LocalHubWebSocket[]
    ├── Broadcast: {"type":"stats","data":{SystemInfo}}
    ├── Broadcast: {"type":"log","data":{LogEntry}}
    └── Client messages: ping → pong, {} → ack
```

5.2 Request Flow (HTTP)

```
Client → LocalHubServer.serveHttp(session)
      │
      ├── URI starts with "/api/" → ApiRouter.handle(session)
      │   ├── Method = GET → handleGet(path) → JSON response
      │   ├── Method = POST → handlePost(session, path) → JSON response
      │   └── Else → 405 Method Not Allowed
      │
      ├── URI starts with "/ws" → Info response (use WebSocket protocol)
      │
      ├── ModuleManager.findHandler(session, uri) → first non-null module response
      │   └── Iterates enabled modules, calls handleRequest(session, path)
      │
      └── AssetServer.serve(uri) → static file from assets
          └── Returns null → 404 Not Found
```

5.3 Request Flow (WebSocket)

```
Client → HTTP Upgrade request → LocalHubServer.openWebSocket(handshake)
      │
      └── LocalHubWebSocket created
          ├── onOpen()
          │   ├── Register with WebSocketHub
          │   ├── Add to ClientTracker (protocol = "WebSocket")
          │   ├── Log connection
          │   └── Send {"type":"welcome","clientId":"<UUID>"}
          │
          ├── onMessage(frame)
          │   ├── "ping" → {"type":"pong","ts":<timestamp>}
          │   ├── JSON object → {"type":"ack","echo":<original>}
          │   └── Text → {"type":"ack","message":"<text>"}
          │
          ├── onClose(code, reason, byRemote)
          │   ├── Unregister from WebSocketHub
          │   ├── Remove from ClientTracker
          │   └── Log disconnection
          │
          └── onException(exception)
              ├── Log error
              └── detach() (same as onClose cleanup)
```

5.4 State Flow Graph

```
SystemInfoProvider.collectSnapshot()
      │
      ├── Updates lastSnapshot: StateFlow<SystemInfo?>
      │    └── Consumed by: DashboardViewModel → DashboardScreen (Compose)
      │    └── Consumed by: ApiRouter (REST API responses)
      │    └── Consumed by: LocalHubService (WebSocket broadcast)
      │
      └── Called every STATS_INTERVAL_MS (2000ms) by LocalHubService

  AppLogger.log() / logAsync()
      │
      ├── Persists to Room via LogRepository
      │    └── Consumed by: LogsViewModel → LogsScreen (Compose)
      │    └── Consumed by: ApiRouter /api/logs
      │
      └── Emits to logs: SharedFlow<LogEntry>
          └── Consumed by: DashboardViewModel (recent 40 logs)
          └── Consumed by: LocalHubService → broadcastLog() → WebSocket

  ClientTracker
      │
      └── clients: StateFlow<List<ConnectedClient>>
          └── Consumed by: ClientsViewModel → ClientsScreen (Compose)
          └── Consumed by: ApiRouter /api/clients
          └── Consumed by: DashboardViewModel (client count)
```

5.5 Foreground Service Lifecycle

```
SettingsViewModel.startServer()
      │
      └── startForegroundService(Intent with ACTION_START)
          │
          └── LocalHubService.onStartCommand()
              │
              ├── startServer()
              │   ├── server.startServer()
              │   ├── startForeground(notification)
              │   ├── acquireWakeLock(10 min timeout)
              │   └── Launch collection coroutine:
              │       ├── Collect logger.logs → broadcastLog(entry)
              │       └── Loop every 2000ms:
              │           ├── collectSnapshot()
              │           ├── broadcastStats(snapshot)
              │           ├── pruneClients()
              │           └── Insert StatisticsSnapshot to Room
              │
              ├── ACTION_STOP → stopServer() + stopForeground() + stopSelf()
              ├── ACTION_APPLY_PORT → setPort() + restartServer(newPort)
              ├── ACTION_SAVE_NAME → setServerName()
              └── ACTION_CLEAR_LOGS → logRepository.clear()
```

5.6 Navigation Structure (Android App)

```
MainActivity
      │
      └── LocalHubNavHost (Bottom Navigation)
          ├── DashboardScreen (route: "dashboard")
          │   └── DashboardViewModel
          │       ├── systemInfo: StateFlow<SystemInfo?>
          │       ├── serverStatus: StateFlow<ServerStatus>
          │       ├── clients: StateFlow<List<ConnectedClient>>
          │       └── logs: StateFlow<List<LogEntry>>
          │
          ├── LogsScreen (route: "logs")
          │   └── LogsViewModel
          │       ├── logs: StateFlow<List<LogEntry>>
          │       └── clear()
          │
          ├── ModulesScreen (route: "modules")
          │   └── ModulesViewModel
          │       ├── modules: StateFlow<List<ModuleInfo>>
          │       └── toggle(module)
          │
          ├── ClientsScreen (route: "clients")
          │   └── ClientsViewModel
          │       └── clients: StateFlow<List<ConnectedClient>>
          │
          └── SettingsScreen (route: "settings")
              └── SettingsViewModel
                  ├── running: StateFlow<Boolean>
                  ├── serverStatus: StateFlow<ServerStatus>
                  ├── port: StateFlow<Int>
                  ├── theme: StateFlow<String>
                  ├── autostart: StateFlow<Boolean>
                  ├── serverName: StateFlow<String>
                  ├── startServer()
                  ├── stopServer()
                  ├── applyPort(port)
                  ├── setTheme(theme)
                  ├── setAutostart(enabled)
                  ├── setServerName(name)
                  └── clearLogs()
```

5.7 Browser Dashboard Architecture

```
Browser → http://<device-ip>:8080/
      │
      ├── GET / → index.html (served by AssetServer)
      │
      ├── GET /style.css → style.css (served by AssetServer)
      │
      ├── GET /app.js → app.js (served by AssetServer)
      │
      └── WebSocket → ws://<device-ip>:8080/ws
          │
          └── Real-time updates:
              ├── {"type":"stats","data":{SystemInfo}} → updateSystem()
              └── {"type":"log","data":{LogEntry}} → addLog()
```

The dashboard uses vanilla JavaScript (no framework). It:

- Fetches initial data via REST API (`/api/system`, `/api/device`, `/api/modules`, `/api/logs`, `/api/clients`)
- Maintains a WebSocket connection for real-time updates
- Auto-reconnects on WebSocket close (2-second delay)
- Polls static data every 5 seconds as a fallback
- Caps the log display at 120 entries (removes oldest)
- Uses CSS custom properties for theming
- Is fully responsive (mobile-first grid layout)

---

6. ANDROID APPLICATION RESPONSIBILITIES

6.1 Purpose

The Android application is the **control plane** for LocalHub. It is where the user starts and stops the server, changes settings, views system telemetry, inspects logs, manages modules, monitors connected clients, and — in future phases — interacts with the assistant, manages files, and controls media.

6.2 Responsibilities

Responsibility	Current State	Future State
Server lifecycle management	✅ Implemented	Stable
System telemetry display	✅ Implemented	Add charts, historical data
Log viewer	✅ Implemented	Add filtering, search, export
Module management	✅ Implemented	Add plugin installation, configuration
Client monitoring	✅ Implemented	Add client details, ban/block
Settings management	✅ Implemented	Add auth settings, assistant settings
Assistant interaction	❌ Not started	Chat interface, voice input
File management	❌ Not started	Browse, upload, download, delete
Media control	❌ Not started	Playback UI, playlist, equalizer
Automation management	❌ Not started	Rule creation, scheduling, monitoring

6.3 UI Architecture Rules

1. **Single Activity.** `MainActivity` is the only Activity. All screens are Compose composables navigated via `NavController`.
2. **Bottom Navigation.** Five destinations: Dashboard, Logs, Modules, Clients, Settings. New screens are nested within these or accessed via sub-navigation.
3. **ViewModel per Screen.** Each screen has exactly one ViewModel. ViewModels extend `LocalHubViewModel` (which provides `container: AppContainer`).
4. **State collection.** All state is collected via `collectAsStateWithLifecycle()` from the `lifecycle-runtime-compose` library.
5. **Theme.** Material 3 with custom `LocalHubColors` exposed via `CompositionLocal`. Dark-first. Supports dark, light, and system modes.
6. **No direct server access from UI.** The UI never talks to `LocalHubServer` directly. It reads state from `AppContainer`'s `StateFlow` properties.

6.4 Compose Component Rules

- **Reusable components** live in `ui/components/` (e.g., `StatCard`, `LogItem`, `ClientCard`, `ModuleCard`, `ServerStatusCard`, `SectionHeader`, `InfoRow`).
- **Screen-specific components** live alongside their screen in `ui/screen/`.
- **Every composable** should accept state as parameters, not reach into ViewModels or the AppContainer directly (except screen-level composables that create their ViewModel).
- **Previews** should be provided for all reusable components.
- **Content descriptions** must be provided for all icons and interactive elements.

6.5 Screen Creation Checklist

When creating a new screen:

1. Create `ui/screen/NewScreen.kt` with the `@Composable fun NewScreen(viewModel: NewViewModel = viewModel())`.
2. Create `ui/viewmodel/NewViewModel.kt` extending `LocalHubViewModel`.
3. Add the route to `LocalHubNavHost` as a new `composable()` entry.
4. If it's a top-level destination, add it to the `Screen` sealed class and the `NavigationBar`.
5. If it's a sub-screen, add it to the appropriate parent's navigation graph.

---

7. BROWSER UI RESPONSIBILITIES

7.1 Purpose

The Browser UI is the **remote control plane** for LocalHub. It is accessed by any device on the same network via a web browser. It provides the same monitoring and management capabilities as the Android app but through a responsive web interface.

7.2 Responsibilities

Responsibility	Current State	Future State
System telemetry display	✅ Implemented	Add historical charts, alerts
Live log streaming	✅ Implemented	Add filtering, search, export
Module listing	✅ Implemented	Add configuration, enable/disable
Client listing	✅ Implemented	Add details, management
Device information	✅ Implemented	Stable
Assistant chat	❌ Not started	Full chat interface
File management	❌ Not started	Browse, upload, download
Media control	❌ Not started	Player UI, playlist
Automation management	❌ Not started	Rule creation, monitoring
Authentication	❌ Not started	Login, session management

7.3 Technology Constraints

1. **No frameworks.** The browser UI uses vanilla JavaScript (ES5+ for maximum compatibility), vanilla CSS (no preprocessors), and vanilla HTML. No React, no Vue, no Angular, no Svelte.
2. **No build step.** The HTML, CSS, and JS files are served directly from `app/src/main/assets/dashboard/`. They must work when opened directly — no compilation, no bundling, no transpilation.

3. **No external CDN.** Every dependency must be bundled within the assets folder. The UI works completely offline.
4. **Dark-first aesthetic.** The design language is "dark glass" — translucent surfaces, subtle borders, accent colors (teal `#22D3A6`, blue `#38BDF8`), and a radial gradient background.
5. **Responsive design.** The dashboard must work on screens from 320px (small phone) to 2560px (large desktop). CSS Grid with `auto-fit` and `minmax()` is the primary layout mechanism.
6. **WebSocket-first, polling-fallback.** The dashboard connects via WebSocket for real-time updates but polls REST endpoints as a fallback.

7.4 File Organization

```
app/src/main/assets/dashboard/
  ├── index.html      — Page structure
  ├── style.css       — All styles (single file, no imports)
  └── app.js          — All JavaScript (IIFE pattern, "use strict")
```

As the browser UI grows, the structure will evolve:

```
app/src/main/assets/dashboard/
  ├── index.html
  ├── style.css
  ├── app.js
  ├── pages/
  │   ├── assistant.html
  │   ├── files.html
  │   ├── media.html
  │   └── automation.html
  ├── css/
  │   ├── components.css
  │   └── pages.css
  ├── js/
  │   ├── api.js          — REST client wrapper
  │   ├── ws.js           — WebSocket manager
  │   ├── router.js       — Client-side page routing
  │   └── utils.js        — Shared utilities
  └── icons/
      └── *.svg
```

7.5 JavaScript Architecture Rules

1. **IIFE Pattern.** Each JS file wraps in `(function() { "use strict"; ... })();` to avoid global scope pollution.

2. **No global variables.** All state is encapsulated within closures.
3. **DOM access via `getElementById`.** No `querySelector` for hot paths (performance). Use `getElementById` with cached references.
4. **XSS prevention.** All dynamic content is escaped via `escapeHtml()` before insertion. No `innerHTML` with untrusted data.
5. **Error resilience.** All network calls are wrapped in `.catch()`. All JSON parsing is wrapped in `try/catch`. The dashboard must never show a blank screen due to an error.

---

8. SERVER RESPONSIBILITIES

8.1 Purpose

The LocalHub server is the communication backbone. It is an embedded HTTP/WebSocket server that runs on the Android device, serving the dashboard, exposing the REST API, and streaming real-time updates to connected clients.

8.2 Technology Stack

- **NanoHTTPD 2.3.1** — Lightweight, embeddable HTTP server.
- **NanoWSD 2.3.1** — WebSocket extension for NanoHTTPD (same version, API compatibility guaranteed).
- **Gson 2.10.1** — JSON serialization/deserialization.
- **Kotlin Coroutines** — For structured concurrency in the service layer.

8.3 Server Architecture

```
LocalHubServer extends NanoWSD
      │
      ├── ApiRouter
      │    ├── Handles all /api/* routes
      │    ├── GET endpoints return JSON from repositories/system info
      │    ├── POST endpoints process request bodies
      │    └── All responses include CORS headers
      │
      ├── AssetServer
      │    ├── Serves dashboard HTML/CSS/JS from app assets
      │    ├── Maps / → index.html
      │    ├── Maps /style.css → style.css
      │    ├── Maps /app.js → app.js
      │    └── Sets Cache-Control: no-cache
      │
      ├── WebSocketHub
      │    ├── Maintains CopyOnWriteArrayList<LocalHubWebSocket>
      │    ├── broadcast(json) sends to all connected clients
      │    └── Thread-safe for concurrent sends
      │
      └── LocalHubWebSocket (per connection)
          ├── Registers with WebSocketHub on open
          ├── Tracks in ClientTracker on connect
          ├── Handles ping/pong, echo/ack
          └── Cleans up on close/error
```

8.4 Server Responsibilities

1. **Accept HTTP connections** on the configured port (default 8080).
2. **Route requests** to the appropriate handler (API, module, asset, or 404).
3. **Serve the dashboard** from app assets with correct MIME types.
4. **Process REST API calls** and return JSON responses.
5. **Accept WebSocket upgrades** and manage connections.
6. **Broadcast real-time updates** (system stats, log entries) to all connected WebSocket clients.
7. **Track connected clients** (both HTTP and WebSocket) with their IP, user-agent, protocol, connect time, and last activity.
8. **Prune stale HTTP clients** after `HTTP_CLIENT_TTL_MS` (60 seconds).
9. **Log all significant events** (server start/stop, client connect/disconnect, errors).

8.5 Server Configuration

Setting	Current	Source
Port	8080	`PreferencesManager.portFlow`
Server Name	"LocalHub"	`PreferencesManager.serverNameFlow`
Stats Interval	2000ms	`Constants.STATS_INTERVAL_MS`
HTTP Client TTL	60s	`Constants.HTTP_CLIENT_TTL_MS`
Log Retention	7 days	`Constants.LOG_RETENTION_DAYS`
Wake Lock Timeout	10 min	`LocalHubService.acquireWakeLock()`
NanoHTTPD Timeout	30s	`start(30_000, false)`

8.6 CORS Policy

Currently, all responses include `Access-Control-Allow-Origin: *`. This is acceptable for the LAN-only use case but must be restricted when authentication is implemented:

- In development (LAN only): `*` is acceptable.
- In production (with auth): CORS must be restricted to the server's own origin.
- API responses with authentication must include appropriate `Access-Control-Allow-Credentials: true` headers.

---

9. ASSISTANT ARCHITECTURE

9.1 Overview

The LocalHub Assistant is an on-device, offline-capable intelligent agent that understands natural language commands, maintains conversational context, executes device actions, manages files, controls media, and automates repetitive tasks. It is the crown jewel of LocalHub — the feature that transforms it from a monitoring tool into a true local operating system.

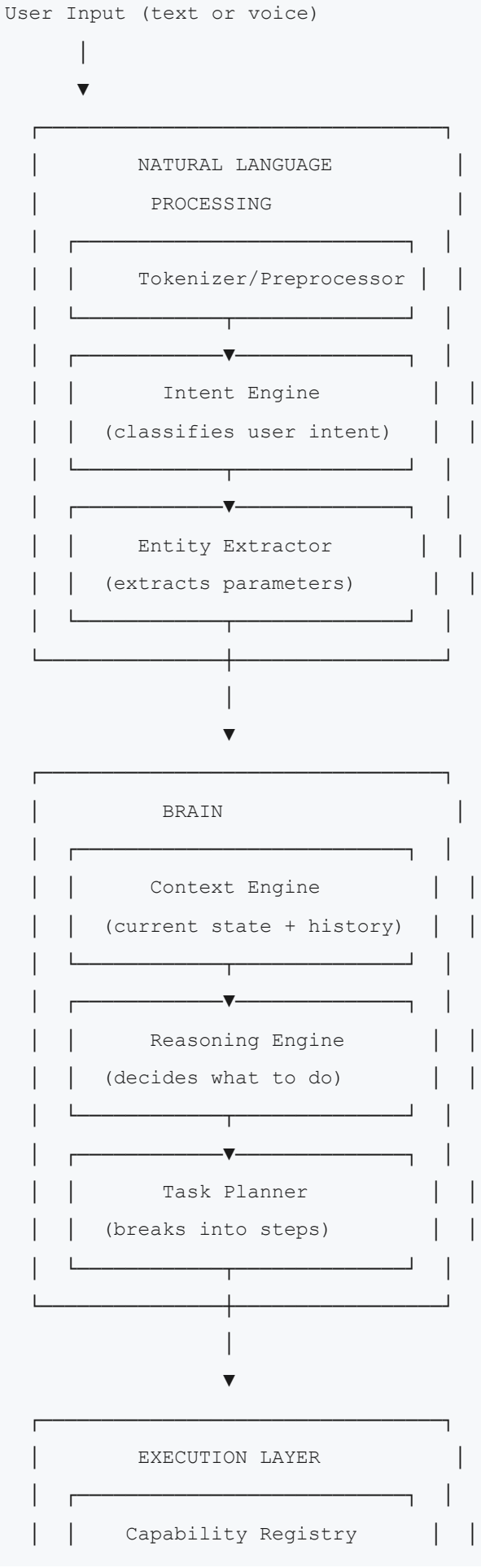
9.2 Design Philosophy

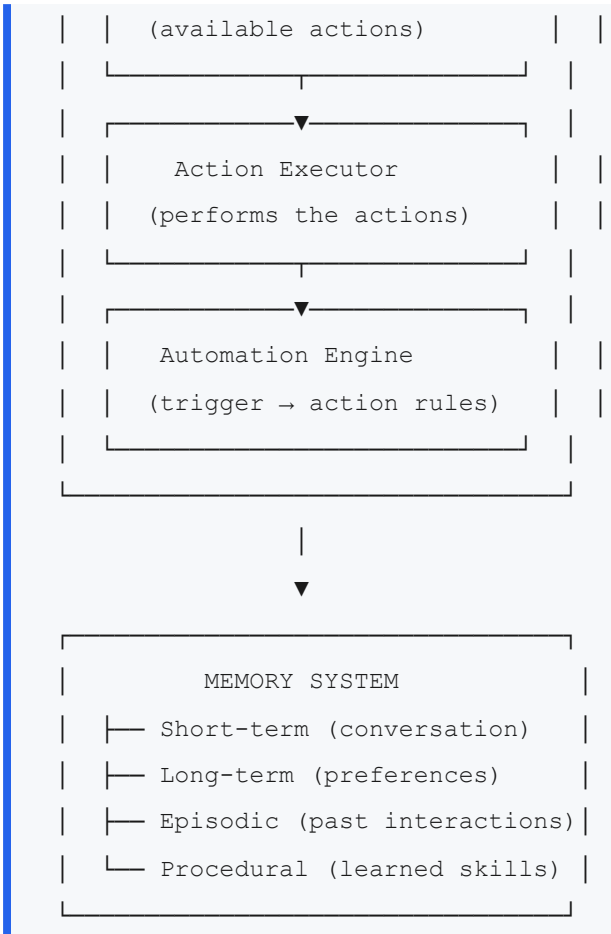
The Assistant must be:

1. **Fully offline.** No cloud LLM calls. No external API dependencies. All reasoning runs on-device using local models or rule-based logic.
2. **Contextually aware.** The Assistant knows what device it's running on, what the server status is, what modules are active, what files exist, and what the user has been asking about.
3. **Deterministic in execution.** While natural language understanding may have probabilistic elements, the execution of recognized intents must be deterministic. "Turn on the server" must always start the server. It must never "sometimes" start the server.

4. **Transparent in reasoning.** The Assistant must be able to explain what it did and why. Every action is logged. Every decision has a reason.
5. **Role-aware.** The Assistant inherits the user's login session and can only perform actions that the user's role permits.
6. **Secure.** The Assistant cannot bypass authentication. It cannot access files outside permitted paths. It cannot execute commands that the user's role forbids.

9.3 High-Level Architecture




(available actions)
Action Executor
(performs the actions)
Automation Engine
(trigger → action rules)
MEMORY SYSTEM
Short-term (conversation)
Long-term (preferences)
Episodic (past interactions)
Procedural (learned skills)

9.4 Package Structure for Assistant

```
com.localhub.localhub.assistant/
  ├── AssistantManager.kt          — Main coordinator
  ├── AssistantSession.kt          — Per-user session state
  │
  ├── brain/
  │   ├── Brain.kt                 — Central reasoning engine
  │   ├── ReasoningEngine.kt       — Decision-making logic
  │   └── ThoughtTrace.kt          — Audit trail of reasoning
  │
  ├── context/
  │   ├── ContextEngine.kt         — State aggregation
  │   ├── ContextSnapshot.kt       — Point-in-time context
  │   ├── DeviceContext.kt         — Device-specific context
  │   ├── ServerContext.kt         — Server-specific context
  │   ├── UserContext.kt           — User-specific context
  │   └── ConversationContext.kt   — Conversation history
  │
  ├── intent/
  │   ├── IntentEngine.kt          — Intent classification
  │   ├── Intent.kt                — Intent data class
  │   ├── IntentResult.kt          — Classification result
  │   ├── IntentCategory.kt        — Enum of intent categories
  │   └── EntityExtractor.kt       — Parameter extraction
  │
  ├── capability/
  │   ├── CapabilityRegistry.kt    — Available actions catalog
  │   ├── Capability.kt            — Capability interface
  │   ├── CapabilityDescriptor.kt  — Metadata about a capability
  │   ├── CapabilityResult.kt      — Execution result
  │   └── capabilities/            — Concrete capability implementations
  │       ├── ServerCapability.kt
  │       ├── SystemCapability.kt
  │       ├── FileCapability.kt
  │       ├── MediaCapability.kt
  │       ├── ModuleCapability.kt
  │       ├── LogCapability.kt
  │       └── AutomationCapability.kt
  │
  ├── planner/
  │   ├── TaskPlanner.kt           — Multi-step task planning
  │   ├── Task.kt                  — Task data class
```

```
│   ├── TaskStep.kt              — Individual step
│   ├── TaskResult.kt            — Execution result
│   └── TaskExecutor.kt          — Step-by-step executor
│
├── automation/
│   ├── AutomationEngine.kt      — Trigger → action rules
│   ├── Trigger.kt               — Trigger definition
│   ├── AutomationRule.kt        — Rule data class
│   ├── TriggerType.kt           — Enum (time, event, state)
│   └── Action.kt                — Action definition
│
├── memory/
│   ├── MemoryManager.kt         — Memory orchestration
│   ├── ShortTermMemory.kt       — Conversation buffer
│   ├── LongTermMemory.kt        — Persistent preferences
│   ├── EpisodicMemory.kt        — Past interactions
│   └── ProceduralMemory.kt      — Learned patterns
│
├── nlp/
│   ├── NlpProcessor.kt          — Text preprocessing
│   ├── Tokenizer.kt             — Token splitting
│   ├── PatternMatcher.kt        — Rule-based matching
│   └── SynonymDictionary.kt     — Synonym expansion
│
└── diagnostic/
    ├── AssistantDiagnostics.kt  — Self-monitoring
    ├── PerformanceMonitor.kt    — Latency tracking
    └── ErrorRecovery.kt         — Graceful failure handling
```

9.5 Assistant Lifecycle

```
AssistantManager (initialized in AppContainer)
      │
      ├── On server start:
      │   └── Initialize ContextEngine with current device state
      │   └── Load MemorySystem from Room database
      │   └── Register all Capabilities in CapabilityRegistry
      │   └── Load AutomationRules from database
      │   └── Start AutomationEngine trigger listeners
      │
      ├── On user message:
      │   ├── Preprocess text (NlpProcessor)
      │   ├── Classify intent (IntentEngine)
      │   ├── Extract entities (EntityExtractor)
      │   ├── Build context snapshot (ContextEngine)
      │   ├── Reason about response (Brain)
      │   ├── Plan execution if needed (TaskPlanner)
      │   ├── Execute capabilities (CapabilityRegistry)
      │   ├── Update memory (MemoryManager)
      │   └── Return response to user
      │
      └── On server stop:
          └── Save memory state to Room
          └── Stop AutomationEngine listeners
          └── Clean up resources
```

---

10. BRAIN ARCHITECTURE

10.1 Purpose

The Brain is the central reasoning engine of the Assistant. It receives classified intents, contextual information, and available capabilities, then decides what to do, how to respond, and whether to execute actions.

10.2 Architecture

```
Brain
      │
      ├── Input:
      │    ├── Classified intent (from IntentEngine)
      │    ├── Extracted entities (from EntityExtractor)
      │    ├── Context snapshot (from ContextEngine)
      │    ├── Memory state (from MemoryManager)
      │    └── Available capabilities (from CapabilityRegistry)
      │
      ├── Processing:
      │    ├── Intent routing (which capability handles this?)
      │    ├── Parameter validation (are all required params present?)
      │    ├── Permission check (does this user have authority?)
      │    ├── Conflict resolution (are there contradictory intents?)
      │    ├── Ambiguity resolution (is the intent unclear?)
      │    └── Response generation (what to say back?)
      │
      └── Output:
           ├── Execution plan (which capabilities to invoke, in what order)
           ├── Response text (what to say to the user)
           └── Thought trace (audit trail for diagnostics)
```

10.3 Reasoning Strategies

The Brain uses a **hierarchical reasoning strategy**:

1. **Pattern Match (Fast Path).** If the intent matches a known pattern with high confidence, route directly to the appropriate capability. No planning needed. Example: "start the server" → `ServerCapability.start()`.
2. **Rule-Based Reasoning (Medium Path).** If the intent requires multiple steps or conditional logic, apply predefined rules. Example: "start the server and tell me the IP" → `ServerCapability.start()` → `SystemCapability.getNetworkInfo()` → compose response.
3. **Task Planning (Slow Path).** If the intent is complex or novel, the Task Planner decomposes it into steps and the Brain orchestrates execution. Example: "set up the server for file sharing" → multiple steps: check storage, start server, configure module, verify access.
4. **Fallback (Safety Path).** If the Brain cannot determine what to do, it responds honestly: "I'm not sure how to help with that. Can you rephrase?" It never guesses or hallucinates actions.

10.4 Thought Trace

Every reasoning cycle produces a `ThoughtTrace`:

```
data class ThoughtTrace(
      val timestamp: Long,
      val inputText: String,
      val classifiedIntent: IntentResult,
      val contextSnapshot: ContextSnapshot,
      val reasoningSteps: List<ReasoningStep>,
      val selectedCapabilities: List<String>,
      val executionPlan: Task?,
      val responseText: String,
      val confidence: Float,
      val reasoningPath: String // "pattern_match" | "rule_based" | "task_planning" |
  "fallback"
  )

  data class ReasoningStep(
      val step: Int,
      val description: String,
      val input: String,
      val output: String,
      val durationMs: Long
  )
```

The ThoughtTrace is persisted in Room for diagnostics and debugging. It allows the user (and developers) to understand exactly why the Assistant made a particular decision.

10.5 Confidence Thresholds

Confidence	Action
≥ 0.9	Execute immediately (Pattern Match)
0.7 – 0.9	Execute with confirmation (Rule-Based)
0.5 – 0.7	Ask clarifying question
< 0.5	Fall back to "I don't understand"

---

11. CONTEXT ENGINE

11.1 Purpose

The Context Engine aggregates all available information about the current state of the system, the user, and the conversation into a single, coherent `ContextSnapshot` that the Brain and other components can query.

11.2 Context Sources

```
ContextEngine
      │
      ├── DeviceContext
      │   ├── Current system info (from SystemInfoProvider)
      │   ├── Battery status
      │   ├── Network connectivity
      │   ├── Storage availability
      │   └── Screen state (on/off)
      │
      ├── ServerContext
      │   ├── Server running/stopped
      │   ├── Current port
      │   ├── Server name
      │   ├── Connected client count
      │   ├── Active modules
      │   └── Recent log entries
      │
      ├── UserContext
      │   ├── Current user role (admin/web_user)
      │   ├── User preferences
      │   ├── Past interaction patterns
      │   └── Permissions
      │
      └── ConversationContext
          ├── Current conversation history (last N turns)
          ├── Active topic
          ├── Pending confirmations
          └── Unresolved references (pronouns, etc.)
```

11.3 Context Snapshot

```
data class ContextSnapshot(
      val timestamp: Long,
      val device: DeviceContext,
      val server: ServerContext,
      val user: UserContext,
      val conversation: ConversationContext,
      val activeModules: List<String>,
      val recentLogs: List<LogEntry>,
      val environmentFlags: Map<String, Boolean>
  )
```

11.4 Context Update Policy

- **Device context** updates every 2 seconds (aligned with `STATS_INTERVAL_MS`).
- **Server context** updates on every state change (server start/stop, client connect/disconnect, module toggle).
- **User context** updates on login, preference change, or permission change.
- **Conversation context** updates on every message exchanged.

11.5 Context Resolution

The Context Engine resolves references in user input using the current context:

- "It" → the last-mentioned entity in the conversation
- "The server" → the LocalHub server instance
- "My files" → the user's permitted file directory
- "Turn it off" → resolve "it" from conversation history

---

12. INTENT ENGINE

12.1 Purpose

The Intent Engine classifies user input into structured intents that the Brain can reason about. It extracts the **what** (intent category), the **parameters** (entities), and the **confidence** (how sure it is).

12.2 Intent Categories

```
enum class IntentCategory {
      // Server control
      SERVER_START,
      SERVER_STOP,
      SERVER_STATUS,
      SERVER_PORT_CHANGE,
      SERVER_RENAME,

      // System queries
      SYSTEM_INFO,
      CPU_INFO,
      MEMORY_INFO,
      STORAGE_INFO,
      BATTERY_INFO,
      NETWORK_INFO,
      DEVICE_INFO,

      // File operations
      FILE_LIST,
      FILE_READ,
      FILE_WRITE,
      FILE_DELETE,
      FILE_UPLOAD,
      FILE_DOWNLOAD,
      FILE_SEARCH,

      // Media control
      MEDIA_PLAY,
      MEDIA_PAUSE,
      MEDIA_STOP,
      MEDIA_NEXT,
      MEDIA_PREVIOUS,
      MEDIA_VOLUME,
      MEDIA_QUEUE,
      MEDIA_PLAYLIST,

      // Module management
      MODULE_LIST,
      MODULE_ENABLE,
      MODULE_DISABLE,
      MODULE_STATUS,
```

```
    MODULE_INSTALL,

    // Log operations
    LOG_VIEW,
    LOG_SEARCH,
    LOG_CLEAR,
    LOG_EXPORT,

    // Client management
    CLIENT_LIST,
    CLIENT_DETAILS,
    CLIENT_BLOCK,

    // Automation
    AUTOMATION_CREATE,
    AUTOMATION_LIST,
    AUTOMATION_ENABLE,
    AUTOMATION_DISABLE,
    AUTOMATION_DELETE,

    // Assistant meta
    ASSISTANT_HELP,
    ASSISTANT_SETTINGS,
    ASSISTANT_MEMORY,
    ASSISTANT_DIAGNOSTICS,

    // Conversational
    GREETING,
    FAREWELL,
    THANK_YOU,
    CONFIRMATION,
    DENIAL,
    UNKNOWN
}
```

12.3 Intent Data Class

```
data class Intent(
      val category: IntentCategory,
      val entities: Map<String, Any>,  // extracted parameters
      val rawText: String,             // original user input
      val confidence: Float            // 0.0 to 1.0
  )
```

12.4 Classification Strategy

The Intent Engine uses a **multi-layer classification approach**:

1. **Keyword Matching (Layer 1).** Fast regex/keyword matching against known patterns. Handles 80% of commands. Example: `"start server"` matches `SERVER_START` with confidence 0.95.
2. **Pattern Templates (Layer 2).** Structural pattern matching. Handles variations. Example: `"turn on the server"`, `"start the server please"`, `"I want the server running"` all match `SERVER_START`.
3. **Semantic Similarity (Layer 3).** For ambiguous inputs, compare against a pre-built vector space of intent examples. This layer is the slowest but handles novel phrasings.
4. **Fallback.** If no layer produces confidence ≥ 0.5, classify as `UNKNOWN`.

12.5 Entity Extraction

After intent classification, the Entity Extractor pulls parameters from the input:

- **Numeric entities:** "set port to 9090" → `{"port": 9090}`
- **Path entities:** "list files in /sdcard/Download" → `{"path": "/sdcard/Download"}`
- **Module entities:** "enable the dashboard" → `{"module_id": "dashboard"}`
- **Time entities:** "remind me in 5 minutes" → `{"delay_ms": 300000}`
- **Boolean entities:** "turn on autostart" → `{"enabled": true}`

---

13. CAPABILITY REGISTRY

13.1 Purpose

The Capability Registry is the catalog of all actions the Assistant can perform. Each capability is a self-contained unit of functionality that the Brain can invoke.

13.2 Capability Interface

```
interface Capability {
      val id: String
      val name: String
      val description: String
      val requiredRole: UserRole  // minimum role to invoke

      /**
       * Describes the parameters this capability accepts.
       */
      fun describeParameters(): List<ParameterDescriptor>

      /**
       * Execute the capability with the given parameters.
       * Returns a CapabilityResult indicating success/failure and output data.
       */
      suspend fun execute(params: Map<String, Any>, context: ContextSnapshot): Capabili
  tyResult

      /**
       * Check if this capability can execute with the given parameters
       * without actually executing.
       */
      fun canExecute(params: Map<String, Any>, context: ContextSnapshot): Boolean
  }

  data class CapabilityResult(
      val success: Boolean,
      val data: Any?,           // result data (varies by capability)
      val message: String,      // human-readable result description
      val error: String? = null // error message if success = false
  )

  data class ParameterDescriptor(
      val name: String,
      val type: ParameterType,  // STRING, INT, BOOLEAN, PATH, DURATION
      val required: Boolean,
      val default: Any? = null,
      val description: String
  )
```

13.3 Capability Registration

Capabilities are registered during `AssistantManager` initialization:

```
class CapabilityRegistry {
      private val capabilities = mutableMapOf<String, Capability>()

      fun register(capability: Capability) {
          capabilities[capability.id] = capability
      }

      fun get(id: String): Capability? = capabilities[id]

      fun findByIntent(category: IntentCategory): List<Capability> {
          // Map intent categories to capabilities
      }

      fun all(): List<Capability> = capabilities.values.toList()
  }
```

13.4 Core Capabilities

Capability ID	Handles	Required Role
`server_control`	Start, stop, restart, status	WEB_USER
`system_info`	CPU, RAM, storage, battery, network, device	WEB_USER
`file_manager`	List, read, write, delete, search, upload, download	ADMIN
`media_player`	Play, pause, stop, next, prev, volume, queue	WEB_USER
`module_manager`	List, enable, disable, install	ADMIN
`log_viewer`	View, search, clear, export	ADMIN
`client_manager`	List, details, block	ADMIN
`automation_control`	Create, list, enable, disable, delete rules	ADMIN
`assistant_meta`	Help, settings, memory, diagnostics	WEB_USER

---

14. TASK PLANNER

14.1 Purpose

The Task Planner decomposes complex user requests into ordered sequences of capability invocations. It is the "planning brain" that turns high-level goals into concrete execution steps.

14.2 Task Model

```
data class Task(
      val id: String,
      val description: String,
      val intent: Intent,
      val steps: List<TaskStep>,
      val status: TaskStatus,
      val createdAt: Long,
      val completedAt: Long? = null,
      val result: TaskResult? = null
  )

  data class TaskStep(
      val index: Int,
      val capabilityId: String,
      val params: Map<String, Any>,
      val dependsOn: List<Int> = emptyList(),  // indices of prerequisite steps
      val status: StepStatus,
      val result: CapabilityResult? = null,
      val retryCount: Int = 0,
      val maxRetries: Int = 3
  )

  enum class TaskStatus { PENDING, RUNNING, COMPLETED, FAILED, CANCELLED }
  enum class StepStatus { PENDING, RUNNING, COMPLETED, FAILED, SKIPPED }
```

14.3 Planning Strategies

1. **Linear Plan.** Steps execute in order. Each step depends on the previous one. Used for simple multi-step tasks.
2. **Dependency Graph.** Steps can execute in parallel if they have no dependencies. Used for complex tasks with independent sub-tasks.
3. **Conditional Plan.** Some steps only execute if previous steps succeed or produce specific results. Used for tasks with branching logic.

14.4 Execution Policy

- Steps execute sequentially by default (safety).
- A failed step triggers retry (up to `maxRetries`).
- If all retries fail, the task fails and the Brain generates an error response.
- The user can cancel a running task at any time.
- All step results are recorded in the Task for diagnostics.

---

15. AUTOMATION ENGINE

15.1 Purpose

The Automation Engine allows users to create rules that automatically trigger actions based on events, time schedules, or state conditions. It is the "if this then that" system of LocalHub.

15.2 Rule Model

```
data class AutomationRule(
      val id: String,
      val name: String,
      val description: String,
      val trigger: Trigger,
      val actions: List<AutomationAction>,
      val conditions: List<Condition> = emptyList(),  // additional guards
      val isEnabled: Boolean = true,
      val createdAt: Long,
      val lastTriggeredAt: Long? = null,
      val triggerCount: Long = 0
  )

  data class Trigger(
      val type: TriggerType,
      val config: Map<String, Any>
  )

  enum class TriggerType {
      TIME,           // cron-like schedule
      EVENT,          // server event (client connect, log level, etc.)
      STATE,          // state threshold (battery < 20%, CPU > 80%, etc.)
      INTERVAL        // periodic (every N minutes)
  }

  data class AutomationAction(
      val capabilityId: String,
      val params: Map<String, Any>,
      val order: Int
  )

  data class Condition(
      val field: String,      // e.g., "battery.level"
      val operator: String,   // e.g., ">", "<", "==", "!="
      val value: Any          // e.g., 20
  )
```

15.3 Trigger Sources

Trigger Type	Source	Examples
TIME	AlarmManager / WorkManager	"Every day at 8 AM"
EVENT	AppLogger SharedFlow, ClientTracker, ServerStatus	"When a client connects"
STATE	SystemInfoProvider StateFlow	"When battery drops below 20%"
INTERVAL	Coroutine delay loop	"Every 5 minutes"

15.4 Execution Model

```
Trigger fires
      │
      ├── Evaluate conditions (all must pass)
      │   └── If any condition fails → skip
      │
      ├── Execute actions in order
      │   └── Each action invokes a Capability
      │
      ├── Log execution (timestamp, rule, result)
      │
      └── Update triggerCount and lastTriggeredAt
```

15.5 Room Persistence

Automation rules are persisted in Room:

```
@Entity(tableName = "automation_rules")
  data class AutomationRuleEntity(
      @PrimaryKey val id: String,
      val name: String,
      val description: String,
      val triggerJson: String,       // serialized Trigger
      val actionsJson: String,       // serialized List<AutomationAction>
      val conditionsJson: String,    // serialized List<Condition>
      val isEnabled: Boolean,
      val createdAt: Long,
      val lastTriggeredAt: Long?,
      val triggerCount: Long
  )
```

---

16. PLUGIN SDK

16.1 Purpose

The Plugin SDK provides a standardized way for third-party developers to extend LocalHub with new capabilities. Plugins can contribute new HTTP routes, new Assistant capabilities, new automation triggers, and new UI components.

16.2 Plugin Contract

Every plugin must implement the `LocalModule` interface (already defined):

```
interface LocalModule {
      val id: String
      val name: String
      val description: String
      val version: String
      var isEnabled: Boolean

      fun start()
      fun stop()
      fun handleRequest(session: IHTTPSession, path: String): Response?
      fun toInfo(): ModuleInfo
  }
```

16.3 Extended Plugin Interface (Future)

For plugins that contribute Assistant capabilities:

```
interface AssistantPlugin : LocalModule {
      /**
       * Return the set of capabilities this plugin provides to the Assistant.
       */
      fun provideCapabilities(): List<Capability>

      /**
       * Return the set of automation triggers this plugin provides.
       */
      fun provideTriggers(): List<TriggerType>

      /**
       * Return natural language patterns the Intent Engine should recognize
       * for this plugin's capabilities.
       */
      fun provideIntentPatterns(): List<IntentPattern>
  }

  data class IntentPattern(
      val patterns: List<String>,    // regex patterns
      val intentCategory: String,    // custom category name
      val pluginId: String
  )
```

16.4 Plugin Auto-Registration

When a plugin is registered with `ModuleManager`:

1. If it implements `LocalModule` only → standard module behavior.
2. If it implements `AssistantPlugin` → additionally:
   - Register its capabilities with `CapabilityRegistry`.
   - Register its triggers with `AutomationEngine`.
   - Register its intent patterns with `IntentEngine`.

16.5 Plugin Sandboxing

Plugins run in the same process but must respect the following constraints:

- Plugins cannot access the Room database directly. They must use repositories.
- Plugins cannot modify `Constants` or `AppContainer` state.
- Plugins cannot access other plugins' internal state.
- Plugins must handle their own exceptions. An unhandled exception in a plugin disables that plugin (not the server).

- Plugins have a 5-second timeout for `handleRequest()`. Timeouts return 504 Gateway Timeout.

---

17. MEMORY SYSTEM

17.1 Purpose

The Memory System gives the Assistant persistent memory across sessions. It remembers user preferences, past conversations, learned patterns, and operational history.

17.2 Memory Types

Type	Purpose	Persistence	Retention
Short-Term Memory	Current conversation context	In-memory	Session duration
Long-Term Memory	User preferences, learned facts	Room database	Permanent (until cleared)
Episodic Memory	Past interactions and outcomes	Room database	30 days (configurable)
Procedural Memory	Learned task patterns	Room database	Permanent

17.3 Short-Term Memory

```
class ShortTermMemory {
      private val conversationHistory = mutableListOf<ConversationTurn>()
      private val activeTopic = AtomicReference<String?>(null)
      private val pendingConfirmations = mutableMapOf<String, PendingConfirmation>()

      fun addTurn(turn: ConversationTurn)
      fun getRecentTurns(n: Int): List<ConversationTurn>
      fun setActiveTopic(topic: String?)
      fun getActiveTopic(): String?
  }

  data class ConversationTurn(
      val role: TurnRole,  // USER or ASSISTANT
      val text: String,
      val timestamp: Long,
      val intent: Intent? = null,
      val entities: Map<String, Any> = emptyMap()
  )
```

17.4 Long-Term Memory

```
@Entity(tableName = "memories")
  data class MemoryEntity(
      @PrimaryKey(autoGenerate = true) val id: Long = 0,
      val key: String,              // unique memory identifier
      val category: String,         // "preference", "fact", "pattern"
      val value: String,            // serialized value
      val confidence: Float,        // how confident the system is
      val createdAt: Long,
      val lastAccessedAt: Long,
      val accessCount: Int
  )
```

17.5 Episodic Memory

```
@Entity(tableName = "episodes")
  data class EpisodeEntity(
      @PrimaryKey(autoGenerate = true) val id: Long = 0,
      val timestamp: Long,
      val userId: String,
      val inputText: String,
      val intentCategory: String,
      val capabilitiesUsed: String,   // JSON array
      val result: String,             // "success" | "failure" | "partial"
      val responseText: String,
      val durationMs: Long
  )
```

17.6 Memory Access Policy

- **Write-through.** Every memory update is persisted immediately.
- **Read-through.** Memory reads check short-term first, then long-term.
- **Decay.** Episodic memories older than 30 days are pruned during maintenance.
- **Consolidation.** Frequently accessed episodic memories may be promoted to long-term memories.

---

18. FILE MANAGER

18.1 Purpose

The File Manager provides file browsing, upload, download, search, and management capabilities through both the Android app and the browser UI. It operates on the device's local filesystem within permitted directories.

18.2 Security Model

- File access is restricted to configurable "permitted directories" (default: `/sdcard/LocalHub/`).
- Path traversal attacks are prevented by canonicalizing all paths and verifying they fall within permitted directories.
- File operations require ADMIN role.
- File uploads are size-limited (configurable, default 100MB).
- File types can be restricted by MIME type whitelist.

18.3 REST API Endpoints (Planned)

Method	Endpoint	Description
GET	`/api/files/list?path=...`	List directory contents
GET	`/api/files/download?path=...`	Download a file
POST	`/api/files/upload`	Upload a file (multipart)
POST	`/api/files/mkdir`	Create directory
POST	`/api/files/delete`	Delete file/directory
POST	`/api/files/rename`	Rename/move file
GET	`/api/files/search?q=...`	Search files by name
GET	`/api/files/info?path=...`	File metadata

18.4 File Module Architecture

```
class FileManagerModule : LocalModule, AssistantPlugin {
      override val id = "file-manager"
      override val name = "File Manager"
      // ...

      // HTTP routes under /api/files/*
      override fun handleRequest(session: IHTTPSession, path: String): Response? {
          if (!path.startsWith("/api/files/")) return null
          // route to appropriate handler
      }

      // Assistant capabilities
      override fun provideCapabilities(): List<Capability> = listOf(
          FileCapability()
      )
  }
```

---

19. MEDIA PLAYER

19.1 Purpose

The Media Player provides audio and video playback capabilities, controllable from both the Android app and the browser UI. It supports streaming over HTTP to browser clients.

19.2 Features

- Play audio files (MP3, AAC, OGG, WAV, FLAC) from local storage.
- Play video files (MP4, WebM) — streamed to browser UI.
- Playlist management (create, add, remove, reorder).
- Playback controls (play, pause, stop, next, previous, seek, volume).
- Queue management (add to queue, clear queue, shuffle).
- Media metadata extraction (title, artist, album, duration, artwork).
- Background audio playback via foreground service.
- Media notification with playback controls.

19.3 Architecture

```
MediaPlayerModule
      │
      ├── MediaLibrary (file scanning, metadata extraction)
      ├── PlaybackEngine (Android MediaPlayer/ExoPlayer wrapper)
      ├── PlaylistManager (queue, shuffle, repeat)
      ├── MediaStreamer (HTTP streaming to browser clients)
      └── MediaSession (Android MediaSession for notifications)
```

19.4 REST API Endpoints (Planned)

Method	Endpoint	Description
GET	`/api/media/status`	Current playback status
POST	`/api/media/play`	Start playback
POST	`/api/media/pause`	Pause playback
POST	`/api/media/stop`	Stop playback
POST	`/api/media/next`	Next track
POST	`/api/media/previous`	Previous track
POST	`/api/media/seek`	Seek to position
POST	`/api/media/volume`	Set volume
GET	`/api/media/library`	Browse media library
GET	`/api/media/stream?id=...`	Stream audio/video
GET	`/api/media/playlist`	Get current playlist
POST	`/api/media/playlist/add`	Add to playlist

---

20. DASHBOARD

20.1 Purpose

The Dashboard is the primary browser-facing interface. It displays real-time system telemetry, connected clients, active modules, device information, and live logs.

20.2 Current Implementation

The current dashboard (v1) is a single-page application built with vanilla HTML/CSS/JS:

- **Metrics Grid:** CPU, Memory, Storage, Battery, Network — each with value, subtitle, and progress bar.
- **Clients Section:** Lists connected clients with IP, protocol, and last activity.
- **Modules Section:** Lists active modules with status and version.
- **Device Section:** Shows device model, manufacturer, Android version, board, hardware.
- **Logs Section:** Live log stream with timestamp, level, and message.
- **WebSocket Status:** Indicator showing connection state (live/connecting/reconnecting).

20.3 Design Language

- **Dark glass aesthetic.** Background: radial gradient from `#16223B` to `#0B1220`.
- **Translucent surfaces.** Cards use `rgba(27, 39, 64, 0.55)` with `backdrop-filter: blur`.
- **Accent colors.** Teal `#22D3A6` for primary accent. Blue `#38BDF8` for secondary.
- **Rounded corners.** 16px border-radius on cards. 999px on progress bars.
- **Typography.** System font stack. 26px metric values. 12px labels.
- **Shadows.** Subtle `0 8px 30px rgba(0, 0, 0, 0.25)` on cards.
- **Responsive.** Grid auto-fits from 180px minimum card width.

20.4 Future Dashboard Evolution

The dashboard will grow with the project:

- **v2:** Add authentication (login page), sidebar navigation, assistant chat panel.
- **v3:** Add file manager view, media player controls, automation management.
- **v4:** Add charts (historical data), alerts, multi-device view.

Each evolution maintains the dark glass aesthetic and vanilla JS approach.

---

21. AUTHENTICATION & AUTHORIZATION

21.1 Purpose

Authentication verifies who the user is. Authorization determines what the user can do. Together they form the security boundary of LocalHub.

21.2 User Roles

Role	Description	Permissions
ADMIN	Full access. Controls server, manages files, configures modules, creates automation rules, manages other users.	Everything.
WEB_USER	Limited access. Can view system info, interact with the assistant, control media, view dashboard.	Read-only + assistant + media.

21.3 Authentication Flow

```
Browser → GET /login
      │
      ├── Returns login page (HTML form)
      │
      └── POST /api/auth/login
          │
          ├── Validate credentials (username + password)
          ├── Check against stored hash (bcrypt/argon2)
          ├── Generate session token (UUID or JWT-like)
          ├── Store session in memory (with expiry)
          ├── Return {"token": "...", "role": "admin"|"web_user"}
          └── Set cookie: session_token=<token>; HttpOnly; SameSite=Strict
```

21.4 Session Management

```
data class Session(
      val token: String,
      val userId: String,
      val role: UserRole,
      val createdAt: Long,
      val lastAccessedAt: Long,
      val expiresAt: Long,
      val ipAddress: String,
      val userAgent: String
  )

  enum class UserRole { ADMIN, WEB_USER }
```

- Sessions expire after 24 hours of inactivity (configurable).
- Sessions are stored in memory (`ConcurrentHashMap<String, Session>`).
- Session validation happens in a middleware layer before API processing.

- The assistant inherits the session of the logged-in user.

21.5 Authorization Middleware

```
class AuthorizationMiddleware(
      private val sessionManager: SessionManager
  ) {
      fun authorize(session: IHTTPSession, requiredRole: UserRole): Boolean {
          val token = extractToken(session) ?: return false
          val userSession = sessionManager.get(token) ?: return false
          if (userSession.isExpired()) return false
          return userSession.role.ordinal >= requiredRole.ordinal
      }
  }
```

21.6 Password Storage

- Passwords are never stored in plaintext.
- Use bcrypt or argon2 for hashing (whichever is available without external dependencies).
- The admin password is set during initial setup.
- Password changes require the current password for verification.

---

22. SECURITY STANDARDS

22.1 Input Validation

- **All user input is untrusted.** Every string from HTTP requests, WebSocket messages, and form submissions is validated and sanitized.
- **Path inputs** are canonicalized and verified against permitted directories.
- **Numeric inputs** are parsed with bounds checking.
- **JSON inputs** are validated against expected schemas.
- **SQL inputs** use parameterized queries only (Room guarantees this).

22.2 Network Security

- The server listens on `0.0.0.0` (LAN-accessible) — this is intentional for the use case.
- CORS is set to `*` for pre-auth (LAN-only). Post-auth, CORS is restricted.
- WebSocket connections require the same session token as HTTP requests (post-auth).
- Rate limiting: 100 requests per minute per IP (post-auth). 20 requests per minute (pre-auth).

22.3 Data Security

- Secrets (passwords, tokens) are stored in `EncryptedSharedPreferences` or Android Keystore.
- Logs may contain sensitive data — log sanitization strips tokens and passwords.
- Statistics data is local-only. No external transmission.
- Session tokens are random UUIDs. Not predictable. Not sequential.

22.4 File Security

- File access is restricted to permitted directories.
- Path traversal (`../`) is blocked by canonicalization check.
- File uploads are scanned for MIME type consistency.
- Maximum upload size is enforced server-side.
- Symbolic links are not followed.

22.5 Security Audit Checklist

Before any release:

- [ ] No hardcoded secrets in source code.
- [ ] All inputs are validated.
- [ ] All endpoints are authenticated (except `/api/auth/login` and static assets).
- [ ] Sessions expire correctly.
- [ ] File access is restricted.
- [ ] Path traversal is blocked.
- [ ] Rate limiting is active.
- [ ] Logs don't contain sensitive data.
- [ ] CORS is properly configured.
- [ ] WebSocket authentication works.

---

23. REST API STANDARDS

23.1 URL Convention

- All API endpoints are prefixed with `/api/`.
- Resource names are lowercase, plural: `/api/modules`, `/api/clients`, `/api/logs`.
- Query parameters use camelCase: `?path=/sdcard&limit=50`.
- No trailing slashes: `/api/status` (not `/api/status/`).

23.2 Response Format

All responses are JSON with `Content-Type: application/json`:

Success:

```
{
      "running": true,
      "version": "1.0.0",
      "port": 8080
  }
```

Collection:

```
{
      "clients": [
          {"id": "...", "ip": "...", "protocol": "WebSocket"},
          {"id": "...", "ip": "...", "protocol": "HTTP"}
      ]
  }
```

Error:

```
{
      "error": "Not found",
      "message": "No such endpoint: /api/foo"
  }
```

23.3 HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful GET, POST
201	Created	Resource created
400	Bad Request	Invalid input
401	Unauthorized	Missing/invalid session
403	Forbidden	Insufficient role
404	Not Found	Unknown endpoint
405	Method Not Allowed	Wrong HTTP method
429	Too Many Requests	Rate limited
500	Internal Server Error	Unexpected error
504	Gateway Timeout	Plugin timeout

23.4 CORS Headers

Every API response includes:

```
Access-Control-Allow-Origin: *
  Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
  Access-Control-Allow-Headers: Content-Type, Authorization
```

Post-authentication, `Access-Control-Allow-Origin` changes from `*` to the specific origin.

23.5 Adding New Endpoints

1. Add the route handler in `ApiRouter.handleGet()` or `ApiRouter.handlePost()`.
2. Return JSON via the `json()` helper.
3. Include CORS headers (the `json()` helper does this automatically).
4. Add authentication check (post-auth phase).
5. Update the README with the new endpoint.
6. Update the browser dashboard if the endpoint is user-facing.

---

24. WEBSOCKET STANDARDS

24.1 Connection Lifecycle

- Clients connect to `ws://<device-ip>:<port>/ws`.

- On connect, the server sends: `{"type":"welcome","clientId":"<UUID>"}`.
- Clients can send: `ping` → receives `{"type":"pong","ts":<timestamp>}`.
- Clients can send: any JSON → receives `{"type":"ack","echo":<original>}`.
- Clients can send: any text → receives `{"type":"ack","message":"<text>"}`.
- On disconnect, the server cleans up the client from `ClientTracker` and `WebSocketHub`.

24.2 Message Envelope

All server-to-client messages follow the envelope format:

```
{
      "type": "<message_type>",
      "data": { ... }
  }
```

Current message types:

Type	Description	Frequency
`welcome`	Sent on connection	Once per connection
`stats`	System telemetry snapshot	Every 2 seconds
`log`	New log entry	On every log event
`pong`	Response to ping	On demand
`ack`	Echo of client message	On demand

24.3 Future Message Types

Type	Description	Phase
`assistant`	Assistant response	Phase 2
`media_status`	Playback state change	Phase 4
`file_event`	File system change	Phase 4
`automation_event`	Automation triggered	Phase 2
`client_event`	Client connect/disconnect	Phase 3
`notification`	System notification	Phase 3

24.4 WebSocket Rules

1. All messages are JSON (or plain text for ping).
2. No binary messages.
3. Messages are fire-and-forget (no delivery confirmation beyond WebSocket protocol).

4. The server broadcasts to ALL connected clients. No per-client filtering (yet).
5. Post-authentication, broadcasts are filtered by role (admin sees more than web_user).
6. The server never sends more than 10 messages per second per client (rate limiting).

---

25. MODULE STANDARDS

25.1 Module Interface

Every module MUST implement `LocalModule`:

```
interface LocalModule {
      val id: String           // unique identifier (kebab-case)
      val name: String         // human-readable name
      val description: String  // one-line description
      val version: String      // semver string
      var isEnabled: Boolean   // runtime toggle

      fun start()              // initialization logic
      fun stop()               // cleanup logic
      fun handleRequest(session: IHTTPSession, path: String): Response?  // HTTP handle
  r
      fun toInfo(): ModuleInfo // metadata for UI/API
  }
```

25.2 Module Lifecycle

```
Registration → start() if isEnabled
      │
      ├── isEnabled = true → start() is called
      ├── isEnabled = false → stop() is called
      │
      ├── handleRequest() is called for every HTTP request
      │   └── Return null to defer to the next module
      │   └── Return Response to handle the request
      │
      └── Unregistration → stop() is called, module removed
```

25.3 Module ID Convention

- Module IDs are kebab-case: `file-manager`, `media-player`, `automation-engine`.

- The `dashboard` ID is reserved for the built-in dashboard module.
- Module IDs must be unique. `ModuleManager.register()` rejects duplicates.

25.4 Module Registration

Modules are registered in `AppContainer`:

```
val moduleManager = ModuleManager().apply {
      register(DashboardModule())      // built-in
      register(FileManagerModule())    // future
      register(MediaPlayerModule())    // future
      register(AutomationModule())     // future
  }
```

25.5 Module Error Handling

- If `handleRequest()` throws, the exception is caught and logged. The next module gets a chance to handle the request.
- If `start()` throws, the module is marked as failed (not enabled). The server continues.
- If `stop()` throws, the exception is logged. The module is still removed from the registry.

---

26. FOLDER STRUCTURE STANDARDS

26.1 Current Structure

```
LocalHub/
  ├── build.gradle                      # Root build config
  ├── settings.gradle                   # Module includes
  ├── gradle.properties                 # Build properties
  ├── gradle/wrapper/                   # Gradle wrapper
  ├── gradlew / gradlew.bat             # Gradle scripts
  ├── README.md                         # Project documentation
  ├── app/
  │   ├── build.gradle                  # App module build config
  │   ├── proguard-rules.pro            # ProGuard rules
  │   └── src/main/
  │       ├── AndroidManifest.xml       # App manifest
  │       ├── java/com/localhub/localhub/
  │       │   ├── LocalHubApplication.kt        # Application class
  │       │   ├── core/                          # DI, constants
  │       │   ├── data/                          # Data layer
  │       │   │   ├── datastore/                 # DataStore preferences
  │       │   │   ├── local/                     # Room database
  │       │   │   │   ├── dao/                   # Data access objects
  │       │   │   │   └── entity/                # Room entities
  │       │   │   └── repository/                # Repository pattern
  │       │   ├── model/                         # Domain models
  │       │   ├── module/                        # Module system
  │       │   ├── server/                        # HTTP/WS server
  │       │   ├── service/                       # Android services
  │       │   ├── system/                        # System providers
  │       │   ├── ui/                            # Compose UI
  │       │   │   ├── components/                # Reusable composables
  │       │   │   ├── navigation/                # Nav graph
  │       │   │   ├── screen/                    # Screen composables
  │       │   │   ├── theme/                     # Theming
  │       │   │   └── viewmodel/                 # ViewModels
  │       │   └── util/                          # Utilities
  │       ├── res/                               # Android resources
  │       │   ├── drawable/                      # Icons
  │       │   └── values/                        # Strings, themes
  │       └── assets/dashboard/                  # Browser UI
  │           ├── index.html
```

```
|           ├── style.css
|           └── app.js
```

26.2 Planned Structure (Full v1.0)

```
java/com/localhub/localhub/
  ├── LocalHubApplication.kt
  ├── core/
  │   ├── AppContainer.kt
  │   └── Constants.kt
  ├── data/
  │   ├── datastore/
  │   │   └── PreferencesManager.kt
  │   ├── local/
  │   │   ├── AppDatabase.kt
  │   │   ├── Mappers.kt
  │   │   ├── dao/
  │   │   └── entity/
  │   └── repository/
  ├── model/
  ├── module/
  ├── server/
  ├── service/
  ├── system/
  ├── assistant/                    # NEW
  │   ├── brain/
  │   ├── context/
  │   ├── intent/
  │   ├── capability/
  │   ├── planner/
  │   ├── automation/
  │   ├── memory/
  │   ├── nlp/
  │   └── diagnostic/
  ├── auth/                         # NEW
  │   ├── AuthenticationManager.kt
  │   ├── SessionManager.kt
  │   ├── AuthorizationMiddleware.kt
  │   └── PasswordHasher.kt
  ├── filemanager/                  # NEW
  │   ├── FileManagerModule.kt
  │   ├── FileScanner.kt
  │   └── PathValidator.kt
  ├── media/                        # NEW
  │   ├── MediaPlayerModule.kt
  │   ├── PlaybackEngine.kt
```

```
│   ├── PlaylistManager.kt
│   └── MediaStreamer.kt
├── ui/
│   ├── components/
│   ├── navigation/
│   ├── screen/
│   ├── theme/
│   └── viewmodel/
└── util/
```

26.3 Package Creation Rules

1. **New top-level packages** (at `com.localhub.localhub.*` level) are created only for major new subsystems (assistant, auth, filemanager, media).
2. **Sub-packages** within an existing package follow the current pattern:
   - `dao/` for DAOs
   - `entity/` for Room entities
   - `repository/` for repositories
   - `model/` for domain models (or keep in the shared `model/` package)
3. **Utility classes** go in `util/` only if they are truly generic. Domain-specific utilities stay within their domain package.

---

27. CODING STANDARDS

27.1 Kotlin Style

- **Indentation:** 4 spaces. No tabs.
- **Line length:** 120 characters maximum.
- **Braces:** Opening brace on the same line. Closing brace on its own line.
- **When expressions:** Use `when` instead of chained `if-else` for 3+ branches.
- **Null safety:** Prefer non-null types. Use `?` only when null is a valid state. Use `runCatching` for expected failures.
- **Immutability:** Prefer `val` over `var`. Prefer `List` over `MutableList` in public APIs. Prefer `data class` over regular classes for models.
- **Visibility:** Default to `private`. Expose only what other classes need.

27.2 Kotlin Patterns

- **Constructor injection:** Dependencies are constructor parameters, not field-injected.

- **Sealed classes for hierarchies:** `sealed class Screen(...)` for navigation. `enum class` for flat sets.
- **Extension functions:** For utility conversions (e.g., `LogEntity.toDomain()`). Keep in `Mappers.kt` or `FormatUtils.kt`.
- **Companion objects:** For constants and factory methods within a class.
- **Scope functions:** Use `apply` for configuration blocks, `let` for null checks, `run` for combined operations.
- **Coroutines:** Always use structured concurrency. Never launch coroutines from non-suspend functions without a dedicated `CoroutineScope`.

27.3 Compose Patterns

- **State hoisting:** State lives in ViewModels. Composables receive state as parameters and emit events as lambdas.
- **`collectAsStateWithLifecycle()`:** Always use this for Flow collection in Compose. Never use `collectAsState()` without lifecycle awareness.
- **`viewModel()`:** Always use the `viewModel()` composable function to obtain ViewModel instances.
- **Preview:** Provide `@Preview` composables for all reusable components.
- **Modifier:** Always accept `Modifier` as the first optional parameter in custom composables.

27.4 JavaScript Patterns

- **Strict mode:** Always `"use strict"` at the top of IIFE.
- **No global variables:** All state is encapsulated in closures.
- **DOM caching:** Cache `getElementById` results for hot-path elements.
- **Event delegation:** Use event delegation for lists of similar elements.
- **Error handling:** Wrap all network calls and JSON parsing in try/catch.

27.5 Error Handling

- **Server code:** Catch all exceptions. Log them. Return appropriate HTTP status codes. Never let an exception crash the server.
- **UI code:** Handle loading, empty, and error states explicitly. Never show a blank screen.
- **ViewModel code:** Use `runCatching { }.onFailure { logger.error(...) }` for operations that might fail.
- **Repository code:** Let exceptions propagate for critical errors. Catch and return defaults for non-critical errors.

---

28. NAMING CONVENTIONS

28.1 Kotlin Naming

Element	Convention	Example
Package	lowercase, dot-separated	`com.localhub.localhub.server`
Class	PascalCase, noun	`LocalHubServer`, `ApiRouter`
Interface	PascalCase, noun/adjective	`LocalModule`, `Capability`
Function	camelCase, verb-first	`handleRequest()`, `collectSnapshot()`
Property	camelCase, noun	`serverStatus`, `connectedClients`
Constant	SCREAMING*SNAKE*CASE	`DEFAULT_PORT`, `STATS_INTERVAL_MS`
Enum value	SCREAMING*SNAKE*CASE	`SERVER_START`, `DEBUG`
Data class	PascalCase, noun	`ConnectedClient`, `SystemInfo`
ViewModel	PascalCase + "ViewModel"	`DashboardViewModel`
Screen	PascalCase + "Screen"	`DashboardScreen`
Entity	PascalCase + "Entity"	`LogEntity`
DAO	PascalCase + "Dao"	`LogDao`
Repository	PascalCase + "Repository"	`LogRepository`

28.2 File Naming

- One public class per file.
- File name matches the public class name: `LocalHubServer.kt` contains `class LocalHubServer`.
- Exception: Small data classes may share a file (e.g., `ClientModels.kt`).

28.3 REST API Naming

- Endpoints: lowercase, hyphenated segments, plural nouns: `/api/clients`, `/api/files`.
- Query parameters: camelCase: `?limit=50&orderBy=timestamp`.
- JSON fields: camelCase: `"connectedAt"`, `"userAgent"`.

28.4 Database Naming

- Table names: lowercase, plural, snake_case: `logs`, `modules`, `automation_rules`.
- Column names: camelCase (Room convention): `timestamp`, `cpuUsage`, `isEnabled`.

---

29. DOCUMENTATION STANDARDS

29.1 KDoc

Every public class, function, and property must have KDoc:

```
/**
   * Tracks the live list of connected and recently-seen clients.
   *
   * WebSocket clients persist until disconnect. HTTP clients are tracked by
   * IP + User-Agent and pruned after [Constants.HTTP_CLIENT_TTL_MS].
   *
   * Changes are published through [clients] as a [StateFlow].
   */
  class ClientTracker { ... }
```

29.2 Inline Comments

- Use `//` for brief inline comments explaining non-obvious logic.
- Use `/* ... */` for multi-line comments explaining algorithms or workarounds.
- Never comment out code. Remove it. Git remembers.
- TODO comments must include the author and date: `// TODO(lastergamer): Add pagination`.

29.3 README

The README.md is the project's front door. It must contain:

- Project description and vision.
- Feature table.
- Architecture diagram.
- Build instructions.
- Usage guide.
- API reference (or link to it).
- Module extension guide.
- License.

29.4 CHANGELOG

Every release includes a CHANGELOG.md entry:

```
## [1.1.0] - 2025-XX-XX

  ### Added
  - File Manager module with browsing and upload capabilities.

  ### Changed
  - Dashboard now shows historical CPU chart.

  ### Fixed
  - WebSocket reconnection no longer duplicates client entries.
```

---

30. TESTING STANDARDS

30.1 Test Types

Type	Tool	Coverage Target
Unit tests	JUnit 5 + MockK	80% of business logic
Integration tests	JUnit 5 + Room in-memory	All repository methods
UI tests	Compose Test	Critical user flows
Server tests	JUnit 5 + OkHttp client	All API endpoints

30.2 Test Naming

```
@Test
  fun `handleRequest returns 200 with JSON for GET status`() { }

  @Test
  fun `register rejects duplicate module id`() { }
```

30.3 Test Structure

Every test follows **Given-When-Then**:

```
@Test
  fun `startServer sets running to true`() {
      // Given
      val server = buildTestServer()

      // When
      val result = server.startServer()

      // Then
      assertTrue(result)
      assertTrue(server.isRunning)
  }
```

30.4 Test Requirements

- **Unit tests** must run in < 100ms each.
- **No network access** in unit tests. Use fakes/mocks.
- **No database** in unit tests. Use in-memory Room for integration tests.
- **Deterministic.** Tests must pass every time. No flaky tests.
- **Independent.** Tests must not depend on execution order.

---

31. PERFORMANCE STANDARDS

31.1 Server Performance

Metric	Target	Measurement
API response time	< 50ms	Time from request to first byte
WebSocket broadcast latency	< 100ms	Time from event to all clients receiving
Concurrent connections	20+	Simultaneous WebSocket clients
Memory overhead	< 50MB	Server heap usage under load
Startup time	< 2s	From server start to first request served

31.2 UI Performance

Metric	Target	Measurement
Frame rate	60fps	Compose recomposition
Screen transition	< 300ms	Navigation animation duration
List scrolling	No jank	1000+ items in LazyColumn
State update	< 16ms	From StateFlow change to UI update

31.3 Performance Rules

1. **No synchronous I/O on the main thread.** Ever. No exceptions.
2. **Lazy initialization.** Don't create objects until they're needed.
3. **Bounded collections.** Lists, maps, and flows have maximum sizes. Unbounded growth is a memory leak.
4. **Efficient serialization.** Use Gson's streaming API for large payloads. Don't serialize entire databases.
5. **Cache appropriately.** System info is cached for 2 seconds. Don't recalculate on every request.
6. **Prune regularly.** Old logs, stale clients, and expired sessions must be cleaned up.

31.4 Profiling

- Use Android Studio Profiler for memory and CPU analysis.
- Use `System.nanoTime()` for benchmarking specific code paths.
- Use the `AssistantDiagnostics` module (when built) for Assistant performance tracking.
- Log warnings for any operation that exceeds its performance target.

---

32. UI STANDARDS

32.1 Android UI (Compose)

- **Material 3.** All components use Material 3 design language.
- **Dark-first.** The default theme is dark. Light and system themes are available.
- **Custom colors.** Extended via `LocalHubColors` CompositionLocal for semantic colors (CPU, memory, storage, battery, network).
- **Typography.** Custom `LocalHubTypography` based on Material 3 type scale.
- **Spacing.** Consistent 16dp padding for screens. 12dp spacing between cards. 8dp spacing within cards.
- **Cards.** All data containers use `Card` with `surfaceVariant` color at 55% alpha.
- **Progress indicators.** `LinearProgressIndicator` for stat cards. 6dp height. Accent color.

32.2 Browser UI

- **Dark glass aesthetic.** Translucent cards on dark radial gradient background.
- **CSS custom properties.** All colors defined as CSS variables in `:root`.
- **Responsive grid.** `grid-template-columns: repeat(auto-fit, minmax(180px, 1fr))`.
- **Mobile-first.** Design for small screens first, enhance for larger ones.
- **No framework.** Vanilla HTML/CSS/JS. No build step.

32.3 Visual Consistency

- Accent colors are identical between Android and Browser UI.
- Card radius is 16px on both platforms.
- Typography scale is harmonized (similar sizes for similar hierarchy levels).
- Icons use Material Icons on Android. CSS/SVG equivalents on Browser.

---

33. ANIMATION STANDARDS

33.1 Android Animations

- **Screen transitions:** Use Navigation Compose default animations. Do not override unless there's a strong reason.
- **State changes:** Use `animate*AsState` for value animations (progress bars, color changes).
- **List items:** Use `AnimatedVisibility` for show/hide. Use default `LazyColumn` item animations.
- **Duration:** 200-300ms for most animations. 150ms for micro-interactions. 500ms for major transitions.
- **Easing:** `FastOutSlowInEasing` for most transitions. `LinearOutSlowInEasing` for entrances.

33.2 Browser Animations

- **CSS transitions only.** No JavaScript animation libraries.
- **Progress bars:** `transition: width 0.4s ease` (already in style.css).
- **Hover states:** `transition: all 0.2s ease` on interactive elements.
- **Page loads:** Fade-in via CSS `@keyframes`.
- **Duration:** Match Android — 200-400ms for most transitions.

33.3 Animation Rules

1. **Respect `prefers-reduced-motion`.** Both Android and Browser must honor accessibility settings.
2. **No animation on data updates.** Live data (stats, logs) updates without animation to avoid visual noise.
3. **Purposeful.** Every animation communicates a state change. No decorative animations.

34. ACCESSIBILITY STANDARDS

34.1 Android Accessibility

- **Content descriptions:** Every icon and interactive element has a `contentDescription`.
- **Touch targets:** Minimum 48x48dp for all interactive elements.
- **Color contrast:** Minimum 4.5:1 for normal text. 3:1 for large text. All `LocalHubColors` are tested for contrast.
- **Screen reader:** Compose semantics are properly set. `semantics { }` blocks for custom components.
- **Dynamic type:** The app respects the system font size setting. No hardcoded text sizes that ignore accessibility.

34.2 Browser Accessibility

- **Alt text:** All images have `alt` attributes.
- **ARIA roles:** Interactive elements have appropriate ARIA attributes.
- **Keyboard navigation:** All functionality is accessible via keyboard (Tab, Enter, Escape).
- **Color contrast:** Match Android standards (4.5:1 minimum).
- **Focus indicators:** Visible focus rings on all interactive elements.
- **Screen reader:** Semantic HTML (`<header>`, `<main>`, `<section>`, `<nav>`).

---

35. RELEASE STANDARDS

35.1 Version Numbering

Follow Semantic Versioning: `MAJOR.MINOR.PATCH`

- **MAJOR:** Breaking changes to public API or architecture.
- **MINOR:** Backward-compatible new features.
- **PATCH:** Backward-compatible bug fixes.

35.2 Release Checklist

Before every release:

- [ ] All tests pass.
- [ ] No known P0 or P1 bugs.
- [ ] README is up to date.
- [ ] CHANGELOG is written.

- [ ] ProGuard/R8 rules are configured for release builds.
- [ ] `versionCode` and `versionName` are incremented.
- [ ] `Constants.SERVER_VERSION` matches `versionName`.
- [ ] Security audit checklist is complete (Section 22.5).
- [ ] Performance targets are met (Section 31).
- [ ] Accessibility audit is complete (Section 34).
- [ ] APK size is reasonable (< 20MB for release build).

35.3 Build Variants

Variant	Minify	Debug	Purpose
debug	false	true	Development
release	true	false	Production

---

36. FUTURE EXPANSION RULES

36.1 The Extension Principle

LocalHub is designed to grow. New features attach to existing interfaces. The core never changes to accommodate a new feature — the new feature conforms to the core.

36.2 Adding a New Subsystem

When adding a completely new subsystem (e.g., Chat, Notifications, Scheduling):

1. **Create a new top-level package** under `com.localhub.localhub.*`.
2. **Define interfaces first.** What does this subsystem expose to the rest of the app?
3. **Register in AppContainer.** Add the subsystem's manager as a `val` property.
4. **Create a Module.** If the subsystem exposes HTTP endpoints, wrap it in a `LocalModule`.
5. **Create a ViewModel + Screen.** If it has a UI, follow the existing pattern.
6. **Add to the dashboard.** If it has browser-facing data, update `app.js`.
7. **Write tests.** Before merging.

36.3 Adding a New API Endpoint

1. Add to `ApiRouter.handleGet()` or `handlePost()`.
2. Authenticate (post-auth phase).
3. Return JSON via `json()` helper.
4. Document in README.
5. Add WebSocket message type if real-time.

36.4 Adding a New Screen

1. Create `ui/screen/NewScreen.kt`.
2. Create `ui/viewmodel/NewViewModel.kt` extending `LocalHubViewModel`.
3. Add route to `LocalHubNavHost`.
4. Add to bottom navigation (if top-level) or sub-navigation (if nested).

36.5 What NOT to Add

- **Cloud dependencies.** LocalHub is offline-first. Period.
- **Third-party SDKs** that require internet (analytics, crash reporting, ads).
- **DI frameworks.** The manual container works.
- **View-based UI.** Compose only.
- **Java source files.** Kotlin only.
- **Multiple modules** (Gradle modules). Single `:app` module. Simpler builds.

---

37. HOW NEW MODULES SHOULD BE CREATED

37.1 Step-by-Step

Step 1: Define the module class.

```
package com.localhub.localhub.mymodule

  import com.localhub.localhub.module.LocalModule
  import com.localhub.localhub.model.ModuleInfo
  import fi.iki.elonen.NanoHTTPD

  class MyModule : LocalModule {
      override val id = "my-module"
      override val name = "My Module"
      override val description = "Does something useful."
      override val version = "1.0.0"
      override var isEnabled = true

      override fun start() {
          // Initialize resources
      }

      override fun stop() {
          // Clean up resources
      }

      override fun handleRequest(session: NanoHTTPD.IHTTPSession, path: String): NanoHT
  TPD.Response? {
          if (!path.startsWith("/api/my-module/")) return null
          // Handle request
          return NanoHTTPD.newFixedLengthResponse(
              NanoHTTPD.Response.Status.OK,
              "application/json",
              """{"ok":true}"""
          )
      }

      override fun toInfo() = ModuleInfo(
          id = id, name = name, description = description,
          version = version, enabled = isEnabled
      )
  }
```

Step 2: Register in AppContainer.

```
val moduleManager = ModuleManager().apply {
      register(DashboardModule())
      register(MyModule())   // ← Add here
  }
```

Step 3: Add persistence (if needed).

If the module needs to persist data:

- Add entity to `data/local/entity/`.
- Add DAO to `data/local/dao/`.
- Add repository to `data/repository/`.
- Register in `AppDatabase`.
- Wire in `AppContainer`.

Step 4: Add UI (if needed).

- Create screen composable.
- Create ViewModel.
- Add to navigation.

Step 5: Write tests.
Step 6: Document.

37.2 Module Anti-Patterns

❌ **Don't** access the AppContainer directly from within a module. Receive dependencies through constructor parameters or the `handleRequest` session.

❌ **Don't** start background threads in `start()` without tracking them for cleanup in `stop()`.

❌ **Don't** return a response for paths you don't handle. Return `null` to defer to the next module.

❌ **Don't** assume your module is the only one. The `ModuleManager` iterates through all enabled modules. Be a good neighbor.

❌ **Don't** hardcode port numbers, paths, or timeouts. Use `Constants` or `PreferencesManager`.

---

38. HOW OLD MODULES SHOULD BE MODIFIED

38.1 The Modification Hierarchy

When you need to change an existing module, follow this hierarchy:

1. **Can I add a new method/property without changing existing ones?** → Do that.

2. **Can I extend the interface with a default implementation?** → Do that.
3. **Can I add a new class that wraps the existing one (Decorator pattern)?** → Do that.
4. **Can I modify the internals without changing the public API?** → Do that, with tests.
5. **Must I change the public API?** → Only if the current API is fundamentally broken. This requires:
    - Documenting why.
    - Providing a migration path.
    - Updating all callers.
    - Bumping the version.

38.2 Safe Modifications

✅ Adding new methods to a class.
✅ Adding new constructor parameters with default values.
✅ Adding new properties with default values.
✅ Improving internal algorithms.
✅ Fixing bugs.
✅ Adding logging.
✅ Improving performance.

38.3 Dangerous Modifications (Require Approval)

⚠️ Changing method signatures.
⚠️ Removing public methods or properties.
⚠️ Changing the `LocalModule` interface.
⚠️ Renaming classes or packages.
⚠️ Changing database schema.
⚠️ Changing REST API response formats.

38.4 Modification Checklist

Before modifying any existing module:

- [ ] I have read the entire module's code.
- [ ] I understand who calls this module's public API.
- [ ] I have identified all callers that will be affected.
- [ ] I have written tests for the current behavior.
- [ ] I have written tests for the new behavior.
- [ ] I have verified that all existing tests still pass.
- [ ] I have documented the change in CHANGELOG.

---

39. HOW DUPLICATE CODE SHOULD BE AVOIDED

39.1 The Search-First Rule

Before writing any function, class, or utility:

1. **Search the codebase** for similar functionality.
2. **Check** `util/` for generic helpers.
3. **Check** `model/` for existing data classes.
4. **Check other modules** for similar implementations.
5. **Only if nothing exists**, write new code.

39.2 Common Duplication Traps

Trap	Solution
Two modules parsing JSON the same way	Use the shared `Gson` instance from `AppContainer`.
Two screens formatting bytes	Use `FormatUtils.formatBytes()`.
Two classes logging the same way	Use `AppLogger`.
Two screens with similar card layouts	Extract to `ui/components/`.
Two modules with similar HTTP response logic	Use shared helper methods.
Two places checking file paths	Use a shared `PathValidator`.

39.3 When Duplication Is Acceptable

Duplication is acceptable ONLY when:

1. **The two implementations are in different abstraction layers** and serve different purposes (e.g., a DTO in the API layer and a domain model in the business layer may have similar fields).
2. **The cost of abstraction exceeds the cost of duplication.** Two similar functions that will never change independently are fine as-is. Don't over-abstract.
3. **The duplication is temporary.** Mark it with `// TODO: Extract to shared utility` and plan the extraction.

39.4 The DRY Enforcement

During code review (or self-review before committing):

- Run a text search for key function names before adding new ones.
- Check if a new data class duplicates an existing one.
- Check if a new utility function duplicates an existing one.
- If similar code exists in 3+ places, it MUST be extracted.

---

40. HOW FUTURE AI SHOULD THINK BEFORE CHANGING ANYTHING

40.1 The Pre-Change Checklist

Before making ANY change to the LocalHub codebase, the AI must:

Phase 1: Understand

- [ ] I have read the complete repository (all files, not just the ones I think are relevant).
- [ ] I have read this Master Prompt in its entirety.
- [ ] I understand the dependency graph of the component I'm changing.
- [ ] I understand who calls the component I'm changing.
- [ ] I understand the data flow through the component I'm changing.

Phase 2: Plan

- [ ] I have identified the minimal set of files that need to change.
- [ ] I have verified that no existing implementation does what I'm about to build.
- [ ] I have designed the change to extend existing interfaces (not replace them).
- [ ] I have considered the impact on:
  - [ ] The REST API (backward compatibility)
  - [ ] The WebSocket protocol (append-only)
  - [ ] The database schema (migration required?)
  - [ ] The UI (does this change the user experience?)
  - [ ] Performance (does this add latency?)
  - [ ] Security (does this create a vulnerability?)
  - [ ] Offline operation (does this work without internet?)

Phase 3: Implement

- [ ] I am following the coding standards (Section 27).
- [ ] I am following the naming conventions (Section 28).
- [ ] I am using the existing patterns (Repository, ViewModel, Module).
- [ ] I am not introducing any new dependencies without justification.
- [ ] I am not blocking the main thread.
- [ ] I am handling errors gracefully.
- [ ] I am logging significant events.

Phase 4: Verify

- [ ] The change compiles without warnings.
- [ ] Existing tests still pass.
- [ ] New tests cover the new behavior.
- [ ] The REST API still returns the same responses for existing endpoints.

- [ ] The WebSocket protocol is unchanged (or only appended to).
- [ ] The UI is accessible (content descriptions, contrast, touch targets).
- [ ] The change works offline.
- [ ] The change is deterministic.

40.2 The Decision Tree

```
I want to make a change.
      |
      ├── Does an equivalent implementation already exist?
      |   ├── YES → Reuse it. Done.
      |   └── NO → Continue.
      |
      ├── Can I extend an existing interface/class?
      |   ├── YES → Extend it. Done.
      |   └── NO → Continue.
      |
      ├── Do I need to modify an existing class?
      |   ├── YES → Is it safe? (See Section 38.2)
      |   |   ├── YES → Modify it (with tests).
      |   |   └── NO → Can I wrap it? (Decorator pattern)
      |   |       ├── YES → Wrap it. Done.
      |   |       └── NO → Escalate. This needs human review.
      |   └── NO → Continue.
      |
      ├── Do I need to create a new class?
      |   ├── YES → Does it follow the existing patterns?
      |   |   ├── YES → Create it. Register it. Test it.
      |   |   └── NO → Why not? Fix the design.
      |   └── NO → Then what am I doing? Rethink.
      |
      └── Does this change violate any rule in Section 4?
          ├── YES → STOP. Redesign.
          └── NO → Proceed with implementation.
```

40.3 The Mental Model

When working on LocalHub, think of the codebase as a **living organism**:

- The **core** (AppContainer, Constants, LocalModule interface) is the **skeleton**. It doesn't change. It supports everything.
- The **modules** are the **organs**. They can be added, removed, or replaced. But the body (skeleton) stays.

- The **data layer** is the **circulatory system**. It flows through everything. Changes to it affect everything.
- The **UI** is the **skin**. It changes most frequently. But it always reflects the state of the organs beneath.
- The **server** is the **nervous system**. It connects everything. Changes to it must be carefully tested.

When you modify one part, ask: **"What else does this affect?"** If you can't answer that question completely, you're not ready to make the change.

40.4 The Anti-Patterns to Avoid

❌ **"I'll just rewrite this class to be cleaner."** → No. If it works, it stays. Improve it incrementally.

❌ **"I'll add a new dependency because it's popular."** → No. Every dependency is a liability. Justify it.

❌ **"I'll change the database schema and use destructive migration."** → Only pre-1.0. After that, proper migrations.

❌ **"I'll add a cloud API because it's easier."** → No. LocalHub is offline-first. Always.

❌ **"I'll skip tests because I'm in a hurry."** → No. Tests are not optional. They are the safety net.

❌ **"I'll make this change 'just for now' and clean it up later."** → No. "Just for now" becomes permanent. Do it right the first time.

❌ **"I'll add a global variable because it's convenient."** → No. State flows through StateFlow. Always.

APPENDIX A: COMPLETE FILE INDEX

Source Files (Kotlin)

File	Package	Responsibility
`LocalHubApplication.kt`	root	Application class, creates AppContainer
`AppContainer.kt`	core	Manual DI container, composition root
`Constants.kt`	core	Global constants
`PreferencesManager.kt`	data.datastore	DataStore preferences
`AppDatabase.kt`	data.local	Room database definition
`Mappers.kt`	data.local	Entity ↔ Domain model mappers
`LogDao.kt`	data.local.dao	Log data access
`ModuleDao.kt`	data.local.dao	Module data access
`SettingDao.kt`	data.local.dao	Setting data access
`StatisticsDao.kt`	data.local.dao	Statistics data access
`LogEntity.kt`	data.local.entity	Log Room entity
`ModuleEntity.kt`	data.local.entity	Module Room entity
`SettingEntity.kt`	data.local.entity	Setting Room entity
`StatisticsEntity.kt`	data.local.entity	Statistics Room entity
`LogRepository.kt`	data.repository	Log data access abstraction
`ModuleRepository.kt`	data.repository	Module data access abstraction
`SettingRepository.kt`	data.repository	Setting data access abstraction
`StatisticsRepository.kt`	data.repository	Statistics data access abstraction
`ClientModels.kt`	model	ConnectedClient data class
`LogModels.kt`	model	LogLevel enum, LogEntry data class
`ModuleModels.kt`	model	ModuleInfo data class
`ServerModels.kt`	model	ServerStatus, StatisticsSnapshot
`SystemModels.kt`	model	CpuInfo, MemoryInfo, StorageInfo, BatteryInfo, NetworkInfo, DeviceInfo, SystemInfo

File	Package	Responsibility
`DashboardModule.kt`	module	Built-in dashboard module
`LocalModule.kt`	module	Module interface contract
`ModuleManager.kt`	module	Module registration and dispatch
`ApiRouter.kt`	server	REST API routing
`AssetServer.kt`	server	Static file serving
`ClientTracker.kt`	server	Connected client tracking
`LocalHubServer.kt`	server	HTTP/WebSocket server
`LocalHubWebSocket.kt`	server	WebSocket connection handler
`WebSocketHub.kt`	server	WebSocket fan-out manager
`BootReceiver.kt`	service	Boot-time autostart
`LocalHubService.kt`	service	Foreground service
`SystemInfoProvider.kt`	system	Device telemetry collection
`MainActivity.kt`	ui	Single Activity entry point
`ClientCard.kt`	ui.components	Client display card
`InfoRow.kt`	ui.components	Label-value row
`LogItem.kt`	ui.components	Log entry display
`ModuleCard.kt`	ui.components	Module display card
`SectionHeader.kt`	ui.components	Section title
`ServerStatusCard.kt`	ui.components	Server status display
`StatCard.kt`	ui.components	System metric card
`LocalHubNavHost.kt`	ui.navigation	Navigation graph
`ClientsScreen.kt`	ui.screen	Clients screen
`DashboardScreen.kt`	ui.screen	Dashboard screen
`LogsScreen.kt`	ui.screen	Logs screen
`ModulesScreen.kt`	ui.screen	Modules screen
`SettingsScreen.kt`	ui.screen	Settings screen

File	Package	Responsibility
`Color.kt`	ui.theme	Brand color definitions
`LocalHubColors.kt`	ui.theme	Extended semantic colors
`Theme.kt`	ui.theme	Material 3 theme
`Type.kt`	ui.theme	Typography definitions
`BaseViewModel.kt`	ui.viewmodel	Base ViewModel with container access
`ClientsViewModel.kt`	ui.viewmodel	Clients screen ViewModel
`DashboardViewModel.kt`	ui.viewmodel	Dashboard screen ViewModel
`LogsViewModel.kt`	ui.viewmodel	Logs screen ViewModel
`ModulesViewModel.kt`	ui.viewmodel	Modules screen ViewModel
`SettingsViewModel.kt`	ui.viewmodel	Settings screen ViewModel
`AppLogger.kt`	util	Logging utility
`FormatUtils.kt`	util	Formatting utilities

Asset Files (Browser UI)

File	Responsibility
`index.html`	Dashboard page structure
`style.css`	Dashboard styles (dark glass aesthetic)
`app.js`	Dashboard logic (REST + WebSocket)

APPENDIX B: DEPENDENCY MANIFEST

Dependency	Version	Purpose
`androidx.core:core-ktx`	1.12.0	Android core extensions
`androidx.lifecycle:*`	2.7.0	ViewModel, lifecycle-aware components
`androidx.compose:*`	BOM 2024.02.02	Jetpack Compose UI
`androidx.navigation:navigation-compose`	2.7.7	Navigation
`androidx.room:*`	2.6.1	Local database
`androidx.datastore:datastore-preferences`	1.1.1	Preferences storage
`com.google.code.gson:gson`	2.10.1	JSON serialization
`org.nanohttpd:nanohttpd`	2.3.1	Embedded HTTP server
`org.nanohttpd:nanohttpd-websocket`	2.3.1	WebSocket support
`org.jetbrains.kotlinx:kotlinx-coroutines-android`	1.7.3	Structured concurrency

Build plugins:

Plugin	Version	Purpose
`com.android.application`	8.2.2	Android build
`org.jetbrains.kotlin.android`	1.9.24	Kotlin compilation
`com.google.devtools.ksp`	1.9.24-1.0.20	Annotation processing (Room)

APPENDIX C: GLOSSARY

Term	Definition
AppContainer	The manual DI container. The composition root of the application.
Browser UI	The web-based interface served by the embedded server.
Capability	An action the Assistant can perform (e.g., start server, list files).
Context Snapshot	A point-in-time aggregation of all available system, user, and conversation state.
Dashboard	The primary browser-facing page showing system metrics and live data.
Deterministic	Producing the same output given the same input. A core design principle.
Intent	A classified user command (e.g., SERVER*START, FILE*LIST).
LocalModule	The interface contract for all pluggable modules.
Module	A self-contained feature unit implementing LocalModule.
Offline-First	The principle that all features work without internet.
Repository	The abstraction layer between domain logic and data sources.
StateFlow	Kotlin's state holder. The primary mechanism for reactive state in LocalHub.
Task	A multi-step plan generated by the Task Planner.
WebSocket Hub	The fan-out mechanism for broadcasting to all WebSocket clients.

APPENDIX D: QUICK REFERENCE CARD

Creating a new module:

1. Implement `LocalModule` interface
2. Register in `AppContainer`
3. Test

Creating a new API endpoint:

1. Add route in `ApiRouter`
2. Return JSON via `json()` helper
3. Add auth check (when implemented)
4. Document

Creating a new screen:

1. Create Screen composable

2. Create ViewModel extending `LocalHubViewModel`
3. Add route to `LocalHubNavHost`

Adding a new preference:

1. Add key in `PrefKeys` object
2. Add flow in `PreferencesManager`
3. Add setter in `PreferencesManager`

Adding a new database table:

1. Create Entity
2. Create DAO
3. Create Repository
4. Add Entity to `AppDatabase`
5. Add DAO to `AppDatabase`
6. Create Mapper functions in `Mappers.kt`
7. Wire in `AppContainer`

Emergency contacts (rules to never break):

- No internet dependency
- No main thread blocking
- No DI framework
- No skipping authentication
- No rewriting working code
- No duplicate implementations

This document is the master prompt for the LocalHub project. It is binding. It is complete. It is the source of truth. Any AI reading this document has everything needed to continue development without losing architectural consistency, introducing regressions, or violating the project's core principles.

Document version: 1.0.0
Last updated: 2026-08-05
Repository: https://github.com/lastergamer8228-spec/LocalHub